

电信客户流失因素识别与挽留策略优化

队伍编号：023

队员姓名：杨张福 刘亚微 胡欣怡

摘要

问题背景：客户流失是一个困扰电信运营商的难题，而获取新客户的成本远高于留住老客户。本文基于一份真实的历史客户数据，进行了探索性数据分析，建立了逻辑回归、随机森林和 XGBoost 三大模型系统研究了客户流失的关键因素并选出了最优模型对客户进行分层和制定差异化挽留策略。

针对问题一，首先利用 pandas 对数据进行了清洗，剔除了 CustomerID、Count、Country 三列与客户流失无关的数据，将 Total Charges 由字符串类型转换为浮点类型便于计算，并按照新客户未产生消费的业务逻辑将产生的 11 条缺失值填充为 0。接着以 Churn Value 为依据统计了整体的客户流失率，然后选取了 Tenure Months、Monthly Charges 和 Total Charges 三个与业务直接相关的特征数据进行了直方图和箱线图可视化分析，观察了各特征数据不同取值变量的流失风险差异，得出在网时长是区分流失与未流失最有效的数值特征。再选取了 8 个分类特征，分别统计流失率并绘制柱状图，发现合同类型、互联网服务类型和支付方式是与客户流失关联较强的 3 个特征。其次，采用 Pearson 相关系数衡量各个数值特征与流失目标的线性关联强度，并以热力图的形式呈现各数值变量之间的相关系数矩阵。最后，对前面的探索性数据分析进行综合，得出合同类型、在网时长和月消费金额等有强区分能力的特征变量应在建立模型时重点考虑。

针对问题二，在数据清洗和探索性数据分析的基础上，首先进行了特征筛选、One-Hot 编码、Z-score 标准化和数据集划分这四个方面的数据准备。接着一步步建立了逻辑回归模型，并通过随机森林和 XGBoost 的对比证明了逻辑回归模型的线性假设的正确性，以及确定了逻辑回归模型作为解决客户流失问题的最优模型。然后引入 SHAP 对逻辑回归的输出进行了归因分析，得到了各个特征的重要性。最后综合 SHAP 分析结果并按平均 SHAP 绝对值对各特征由大到小进行了排序，得到了在网时长、是否有家属（是）、互联网服务（光纤）、合同类型（两年）和总消费金额这五大关键客户流失因素。

针对问题三，根据问题二确定的逻辑回归模型和 SHAP 分析结果，本文首先利用分位数切分法，以模型预测的客户流失概率的第 50 百分位数和第 80 分位数为分界点，将客户划分为低风险客户（小于第 50 百分位数）、中风险客户（第 50 百分位数和第 80 百分位数之间）和高风险客户（第 80 百分位数以上）三个层次。由于大部分客户集中在低流失概率区间，按本文的划分方法可以避免等距划分导致的风险层级过于稀疏。接着，为了检验本文的客户分层是否符合实际的业务区分度，计算了各层次的客户在在网时长、月消费金额、客户生命周期价值（CLTV）、月付合同比例、光纤比例和有家属比例这六个维度的均值或占比。各层次客户在六个维度形成的清晰梯度，说明了本文客户分层策略的合理性和可靠性。然后根据不同客户层次的特点，针对性地制定了差异化客户挽留策略。最后，本文计算了挽留策略的预期收益，得出 1 美元成本可获得 25 美元的可观收入。

关键词：客户流失预测；逻辑回归；SHAP 可解释性；客户分层；差异化挽留策略

1 问题重述

1.1 问题背景

在电信行业，客户流失（Customer Churn）是一个长期困扰运营商的结构性问题。研究表明，获取一位新客户的成本是保留一位现有客户的 5 至 7 倍 [1]，频繁的客户离网不仅侵蚀当期营收，还会持续推高市场获客成本。近年来，机器学习方法在客户流失预测领域得到了广泛的应用，从早期的逻辑回归和决策树，到随机森林 [2]、XGBoost [3] 等集成学习模型，均有大量实证研究。王圣节等 [4] 以国内通信公司实际数据为基础，结合 CatBoost 与 SHAP 构建了可解释预测框架；彭康等 [5] 将遗传算法优化的 XGBoost 与 SHAP 结合，进一步提升了特征归因的可靠性。然而，许多高精度模型以牺牲可解释性为代价，导致预测结果难以向业务方解释和落地。为此，Lundberg 等提出的 SHAP 方法 [6] 为模型输出提供了基于 Shapley 值的归因解释，在电信客户流失场景中已展现出良好的应用潜力 [7, 8]。因此，识别流失风险的关键诱因并在客户流失前进行有效干预，已成为电信企业运营管理的核心议题。

本文所关注的某电信运营商在近一年内客户流失率已由 18% 攀升至 26%，企业亟需基于自身积累的客户历史数据，建立可靠的流失预警机制并制定针对性的挽留策略。为此，本文围绕该企业提供的 7043 条客户记录（涵盖人口统计学信息、账户信息、服务类型及费用明细），通过探索性数据分析、分类建模与可解释性方法，识别客户流失的关键驱动因素，并据此设计差异化的挽留策略，以降低流失率、提升客户生命周期价值。

1.2 问题提出

根据上述业务背景，这一目标可从三个递进的层面逐步达成——首先从历史数据中归纳流失客户的典型特征（问题一），进而建立可量化的预测模型以识别关键驱动因素（问题二），最终基于量化结论制定可落地的差异化挽留方案（问题三）。本文将问题分解为以下三个相互关联的子问题：

问题一：流失客户画像构建。需基于原始客户数据，从多个维度对比流失与未流失客户群体在人口统计学特征、账户服务信息和消费行为上的差异，构建流失客户的核心特征画像，为后续建模提供特征选择依据。

问题二：流失预测模型与关键因素识别。需建立能够准确预测客户流失风险的分类模型，在保证预测精度的前提下，识别驱动客户流失的关键因素并给出可量化的业务解释。

问题三：客户分层与差异化挽留策略。需基于预测结果对客户进行风险分层，针对不同层级客户的差异化特征设计挽留方案，并结合成本收益分析论证方案的经济可行性。

1.3 本文工作概述

本文从三个层面逐步推进。第一，通过探索性数据分析构建流失客户的多维特征画像，识别出在网时长和合同期限为区分力最强的特征。第二，以逻辑回归为基线的三模型对比表明线性假设在该问题上充分，借助 SHAP 归因分析确认前五位关键流失因子（在网时长、家属、光纤、两年合同、总消费），并经由特征消融实验交叉验证。第三，基于分位数分层和 CLTV 评估，提出差异化挽留方案，在保守参数假设下预期投资回报率达 25.4 倍。

2 问题分析

2.1 问题一分析

该问题本质为数据探索与特征分析问题。核心目标是在无先验假设的条件下，通过统计分析和可视化呈现构建流失客户群体的特征画像。需重点关注两方面：一是数据质量——是否存在异常值及类型不一致等影响建模的问题；二是特征区分度——哪些变量在流失与未流失客户之间呈现显著差异。可采用分组统计、分布对比图、相关性分析等手段进行系统探索。

2.2 问题二分析

该问题本质为二分类建模与模型可解释问题。在问题一筛选的特征基础上，构建从客户特征到流失概率的预测模型。模型选择需兼顾精度与可解释性：以逻辑回归为线性基准，以随机森林和 XGBoost 为非线性对照。评估阶段，针对类别不平衡问题采用 AUC 与 F1-score 作为主要指标；分析阶段，引入 SHAP 方法将模型输出分解为各特征的量化贡献，为业务决策提供数据支撑。

2.3 问题三分析

该问题本质为策略优化与投入产出评估问题。基于问题二的流失概率输出，对客户进行风险分层并将层级与客户生命周期价值（CLTV）关联，针对不同层级客户的行为特征设计差异化挽留方案。方案需同时权衡预期收益（减少流失所挽回的收入）与实施成本（优惠力度与运营投入），论证收益与成本之间的经济可行性。

2.4 数据说明

数据来源：题目附件 Telco_customer_churn.xlsx。

数据规模：共 7043 条客户记录，33 个特征变量，涵盖人口统计学特征（性别、年龄、家庭状况）、账户服务信息（在网时长、合同类型、支付方式、服务开通情况）、费用信息（月消费、总消费、生命周期价值）以及流失标签。

数据质量：经检查，数据整体完整无缺失值。其中 Total Charges 列存在 11 条因在网时长为 0 而产生的逻辑空值，已按业务逻辑填充为 0。剔除 CustomerID、Count、Country 三个无信息量列后，剩余 30 个可用特征，全部 7043 条记录完整可用。详细的预处理步骤见第 4 章问题一的数据预处理节。

目标变量分布：流失客户 1869 人（占比 26.54%），未流失客户 5174 人（占比 73.46%），呈现一定程度的类别不平衡，后续建模需采取加权或采样策略加以应对。

2.5 整体建模流程

本文的整体建模流程如图 1 所示。

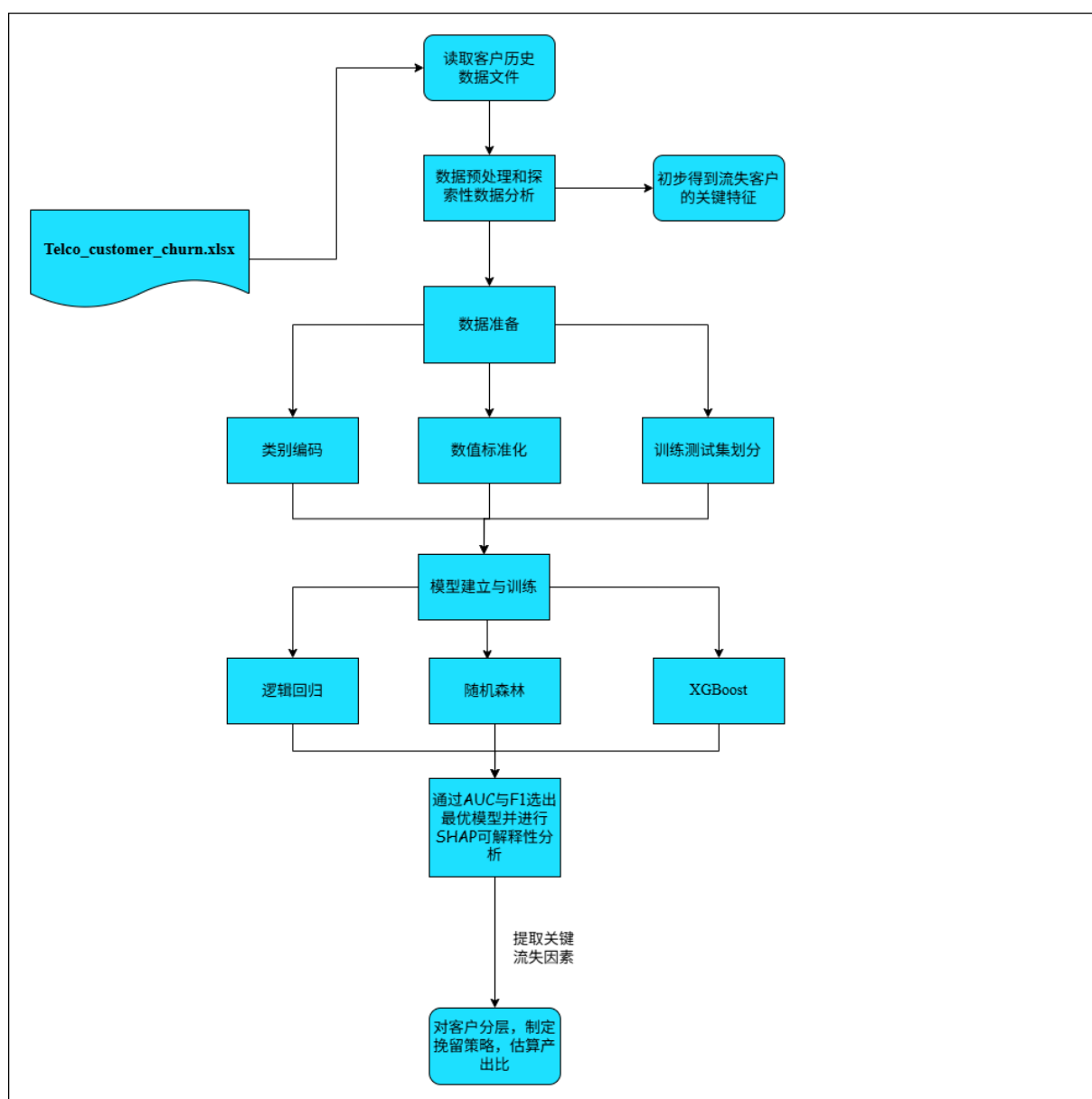


图 1 整体建模流程

首先进行数据预处理与探索性分析，识别数据质量问题和关键特征；然后进行数据准备（类别编码、数值标准化、训练测试集划分），分别训练逻辑回归、随机森林、

XGBoost 三种分类模型并通过 AUC 与 F1-score 进行对比优选；对最优模型进行 SHAP 可解释性分析，提取关键流失因素；最后基于模型输出的流失概率进行客户分层，制定差异化挽留策略并估算投入产出比。

3 模型假设与符号说明

3.1 模型假设

为在可处理的框架内抓住问题的核心矛盾，本文作以下约定：

1. **样本代表性**。所给 7043 条记录能代表该运营商的客户总体，不存在系统性的抽样偏倚。
2. **客户独立应答**。不同客户的离网决策彼此独立，排除同一家庭成员间的联动退订效应。
3. **特征充分性**。现有字段已覆盖客户流失的主要诱因，未被记录的宏观因素（竞争环境、经济波动等）的残余影响较小。
4. **标注可靠**。Churn Value 字段如实记录了客户的留存/离网状态，无误标或漏标。
5. **行为稳态**。考察期内客户的消费模式与服务使用强度未发生剧烈变动，可近似视为平稳。

3.2 符号说明

本文使用的主要符号及其含义如表 1 所示。

4 问题一：流失客户画像构建

4.1 数据预处理

原始数据来自题目附件 Telco_customer_churn.xlsx，共 7043 条记录、33 个字段，涵盖客户人口统计学信息、账户信息、服务订阅信息和流失标签。

首先对数据进行清洗。剔除 CustomerID、Count、Country 三列：CustomerID 作为唯一标识，与客户流失情况无关，无泛化预测能力；Count 恒为 1，方差为 0，对区分客户无贡献；Country 全部为 United States，无差异性，对模型无额外信息量。

其次，经检查 Total Charges 列为字符串类型。经 `pd.to_numeric` 转换后确认 11 条 NaN 记录。追溯发现这 11 条记录的 Tenure Months 均为 0（新客户尚未产生消费），因此填充为 0，符合正常业务逻辑。处理后全部 7043 行无缺失值。

为便于后续分析中引用，表 2 按业务类别整理了关键变量的中英文名称对照及类别归属。后续分析将围绕表中所列的时长与费用、合同与支付、服务订阅、人口与家庭四大维度展开。

表 1 主要符号说明

符号	单位	含义
(一) 数据与特征		
N	—	样本总数 (7043)
p	—	编码后特征维度 (31)
x_i	—	第 i 个样本的特征向量
y_i	—	第 i 个样本的流失标签 (0/1)
r	—	Pearson 线性相关系数
(二) 逻辑回归模型		
P	—	模型输出的流失概率 $\in (0, 1)$
$\hat{y}_i$	—	第 i 个样本的预测类别
β_0	—	截距项 (偏置)
β_j	—	第 j 个特征的回归系数
λ	—	ℓ_2 正则化强度
(三) 评估与验证		
AUC	—	ROC 曲线下面积
F1	—	精确率与召回率的调和均值
k	—	交叉验证折数
(四) 可解释性与分层		
ϕ_j	—	第 j 个特征的 SHAP 值
P_{50}	—	预测概率的第 50 百分位数
P_{80}	—	预测概率的第 80 百分位数
(五) 策略评估		
CLTV	—	客户生命周期价值 (美元)
R_{reach}	—	挽留方案的触达率
R_{retain}	—	触达客户的挽留成功率

表 2 关键变量名称对照表

变量类别	中文名称	原始列名	说明
目标变量	是否流失	Churn Value	1= 流失, 0= 未流失
(一) 时长与费用特征			
数值	在网时长	Tenure Months	单位: 月
数值	月消费金额	Monthly Charges	单位: 美元
数值	总消费金额	Total Charges	单位: 美元
数值	客户生命周期价值	CLTV	Customer Lifetime Value
(二) 合同与支付特征			
分类	合同类型	Contract	月付/一年/两年
分类	支付方式	Payment Method	共 4 种
二分类	是否无纸化账单	Paperless Billing	Yes / No
(三) 服务订阅特征			
多分类	互联网服务类型	Internet Service	DSL/Fiber optic/No
二分类	在线安全服务	Online Security	Yes/No/No internet
二分类	技术支持服务	Tech Support	Yes/No/No internet
(四) 人口与家庭特征			
二分类	是否老年人	Senior Citizen	Yes / No
二分类	是否有配偶	Partner	Yes / No
二分类	是否有家属	Dependents	Yes / No
(五) 其他说明			
文本	客户编号	CustomerID	无预测价值, 已剔除
数值	纬度	Latitude	与流失相关系数 <0.01
数值	经度	Longitude	与流失相关系数 <0.01

4.2 流失客户特征探索

整体流失率

以 Churn Value 为分类依据进行统计, 流失客户 1869 人, 未流失客户 5174 人, 总客户 7043 人, 整体流失率为 26.54%, 如图 2 所示。

客户流失分布

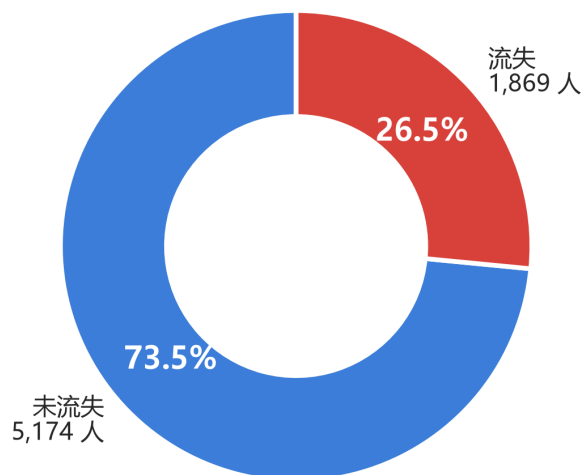


图 2 整体流失率环形图

从分布比例来看，流失与未流失两类样本的比例约为 1:2.8，呈现一定程度的类别不均衡。这一比例虽然不属于严重失衡（如 1:10 以上），但已足以对常规分类模型的训练过程产生影响——模型倾向于将多数类（未流失）识别得更好以降低整体损失，而牺牲对少数类（流失）的识别能力。例如，一个将所有样本都预测为“未流失”的简单模型也能获得 73.46% 的准确率，但这显然不能反映模型在实际业务中的价值。

为此，后续建模中评价指标弃用对类别比例敏感的准确率，转为采用 AUC 和 F1-score；训练时对逻辑回归设置 `class_weight='balanced'` 参数，按类别频数自动调整损失权重，强化对流失客户的关注。

数值特征分布对比

为考察流失与未流失客户在数值特征上的分布差异，选取在网时长（Tenure Months）、月消费金额（Monthly Charges）和总消费金额（Total Charges）三个核心数值特征，分别绘制分组直方图与箱线图，如图 3 所示。

数值特征分布对比

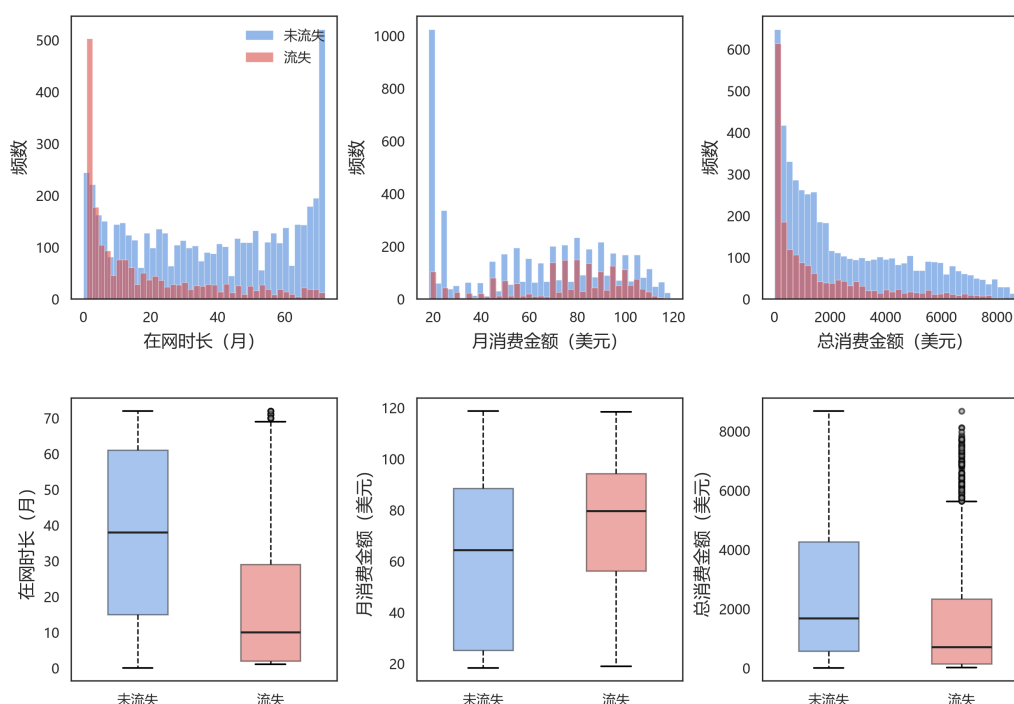


图 3 数值特征分布对比图

由直方图和箱线图可以观测到以下规律：

在网时长：流失客户的分布高度集中于 0-10 个月的低在网时长区间，而未流失客户的分布较为均匀，覆盖 0-72 个月的全区间。箱线图显示两组中位数差距显著，说明在网时长是区分能力最强的数值特征——新客户是流失的高风险群体。

月消费金额：流失客户的月消费整体偏高，在 70-110 美元的高费区间聚集密度明显大于未流失客户。高消费客户对价格更为敏感，流失意愿相对更强。

总消费金额：流失客户的总消费明显偏低且呈左偏分布，但该差异主要源于在网时长长短导致的消费积累不足，属于在网时长的间接效应，其独立区分能力有限。

综合以上分析，在网时长是区分流失与未流失最有效的数值特征。

分类特征流失率对比

为考察分类变量各取值水平下的流失风险差异，选取合同类型、互联网服务类型、支付方式、是否老年人、是否有配偶、是否有家属、是否开通电话服务、是否无纸化账单共 8 个分类特征，分别统计各组流失率并绘制柱状图，如图 4 所示。

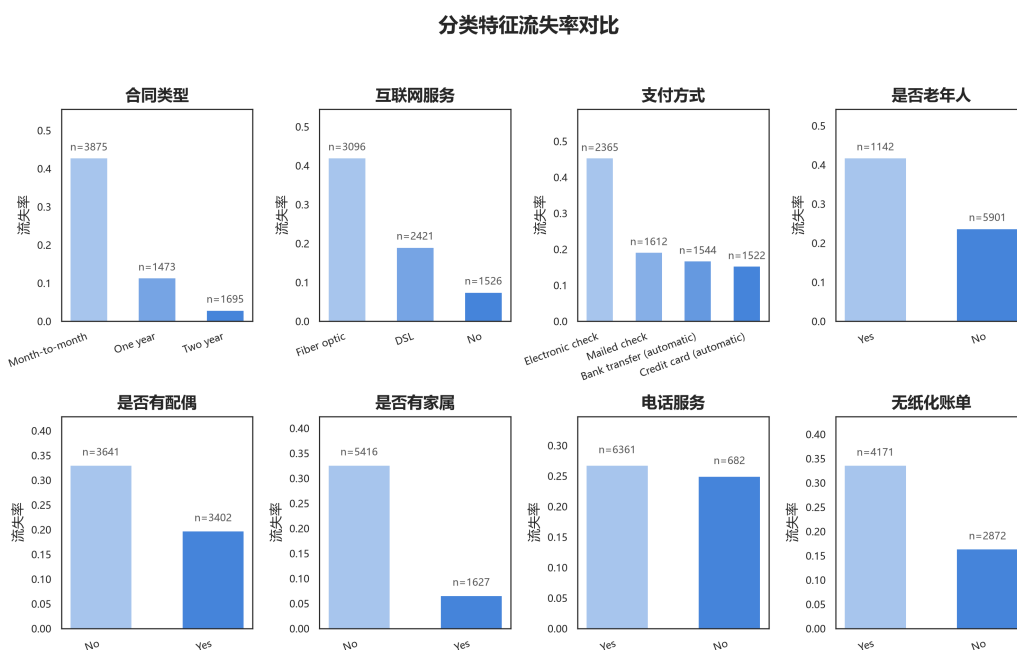


图 4 分类特征流失率对比图

各分类特征的分组流失率如表和图所示，具体分析如下：

合同类型 (Contract)：月付合同的流失率超过 40%，而一年合同约为 10%、两年合同约为 3%。合同期限的差异根本性地改变了客户流失的转换成本——月付客户可以随时解约，而年付和两年合同的客户通常承担了提前解约的违约金或已享受合约折扣，转换运营商的意愿显著降低。

互联网服务类型 (Internet Service)：光纤客户的流失率显著高于 DSL 客户和无互联网客户，可能与光纤服务的高月费有关——高消费群体对服务性价比的要求也更高。

支付方式 (Payment Method)：电子支票客户的流失率约为 45%，而银行自动转账、信用卡自动支付和邮寄支票三种方式的流失率均在 15%-20% 之间，表明支付方式与客户属性之间存在一定的关联性。

此外，有配偶或有家属的客户流失率低于独身客户（家庭绑定提高了离网成本），无纸化账单客户流失率高于纸质账单客户（无纸化账单客户更习惯线上操作，对运营商的服务切换成本感知较低，更容易比价和更换服务商），而是否老年人和是否开通电话服务对流失率的影响相对较小。

相关性热力图

采用 Pearson 相关系数衡量各数值特征与流失目标之间的线性关联强度（取值范围 $[-1, 1]$ ，绝对值越大线性相关越强）。数值变量的相关系数矩阵以热力图形式呈现在图 5 中。

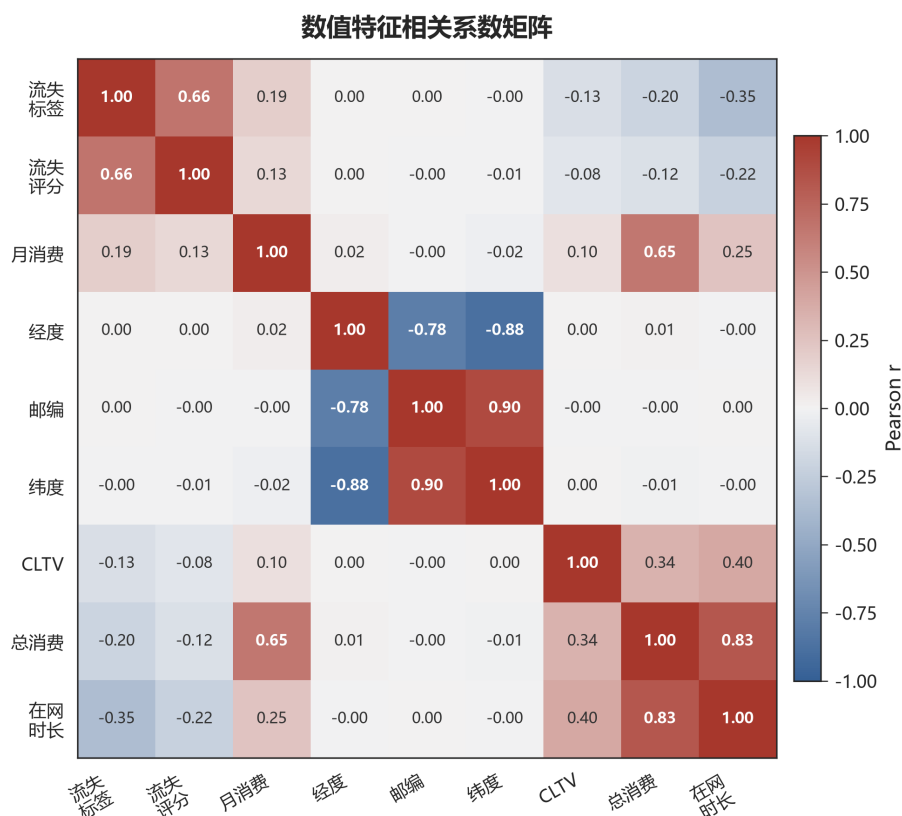


图 5 相关性热力图

由热力图可见：

在网时长：与流失的相关系数为 -0.35 ，在所有数值特征中绝对值最大，呈中等程度的负相关——在网时间越长，流失可能性越低，这与前文分析中“新客户是高危群体”的结论相一致。

月消费金额：与流失的相关系数为 $+0.19$ ，呈弱正相关——月消费越高的客户对服务性价比的要求也越高，流失意愿更强，这与分类特征分析中光纤客户流失率较高的发现互为印证。

总消费金额与 CLTV：与流失的相关系数分别为 -0.20 和 -0.13 ，均为弱负相关，方向符合预期。但值得注意的是，在网时长与总消费之间存在较强的正相关 ($r = 0.83$)，这是因为在网时间越长累计消费自然越高，说明两变量存在多重共线性，后续建模中可考虑择一保留。

地理特征：经度、纬度、邮编三个特征与流失的相关系数绝对值均小于 0.01 ，不具备线性预测价值，说明地理位置与客户的流失决策之间不存在明显的关联关系。

4.3 初步结论

综合上述探索性分析，可从以下维度刻画流失客户的典型特征：在合同层面，流失客户以月付合同为主，其流失率超过 40% ，远高于年付和两年合同客户；在在网时长层面，流失客户高度集中在在网 $0-10$ 个月的新客户群体，在网时长是与流失相关性最强的数值特征 ($r = -0.35$)；在消费层面，流失客户的月消费金额偏高，高消费群体对服

务性价比更为敏感，流失意愿更强；在家庭层面，无配偶、无家属的独身客户流失率高于有家庭绑定的客户，前者离网成本更低。

上述发现为下一节分类模型的建立提供了明确的方向：模型应重点纳入合同类型、在网时长、月消费金额等具有强区分能力的特征变量；而地理信息（经纬度、邮编）因相关系数绝对值均低于 0.01 可预先剔除，从而减少模型维度并降低噪声干扰。

5 问题二：流失预测模型与关键因素识别

在问题一构建的流失客户画像基础上，本节的目标是建立一个能够量化各特征对流失风险的贡献程度的预测模型，以回答题目提出的核心问题：哪些因素是驱动客户流失的关键原因？为此，本文按“数据准备 → 模型建立 → 模型验证与对比 → 关键因子识别”四个阶段依次推进。

5.1 数据准备

在进入建模之前，需将问题一中清洗后的原始数据转换为分类模型可接受的数值格式。按以下三步依次进行。

特征筛选

基于问题一的 EDA 结论和特征自身的性质，剔除四类无建模价值的列：与目标变量等价的冗余列（Churn Label）、基于已知流失信息计算的泄露列（Churn Score，其与 Churn Value 的相关系数高达 0.66）、仅少数样本有值的单边列（Churn Reason），以及在相关性分析中确认与流失无线性关联的地理位置列（Latitude、Longitude、Lat Long、State、City、Zip Code，相关系数均小于 0.01）。共剔除 9 列，保留 20 个候选特征。

类别编码与数值标准化

16 个分类特征中，二分类变量（Gender、Senior Citizen、Partner、Dependents、Phone Service、Paperless Billing）取值仅两种，直接采用 Label 编码映射为 0/1。多分类变量（Contract、Internet Service、Payment Method、Multiple Lines、Online Security、Online Backup、Device Protection、Tech Support、Streaming TV、Streaming Movies）各取值之间不存在天然的序关系——例如合同类型“月付、一年、两年”只是三类不同期限，并非等距的顺序尺度——因此不宜使用 Label 编码，而是采用 One-Hot 编码并设置 `drop_first=True` 以避免多重共线性。编码后特征维度由 20 列扩展为 31 列。

4 个数值特征（Tenure Months、Monthly Charges、Total Charges、CLTV）的量纲和取值范围各不相同。为使各特征在模型训练中得到平等对待、消除量纲差异的影响，本文采用 Z-score 标准化将各数值特征的均值和标准差分别调整为 0 和 1。

数据集划分

按 8:2 比例划分训练集与测试集。划分时设置 `stratify=y`，保证两边的流失率保持一致（均为 26.54%）。最终得到训练集 5634 条、测试集 1409 条，编码后的特征维度为 31 列。

至此，原始数据已转化为包含 31 个标准化特征的数值矩阵，可输入分类模型进行训练和评估。

5.2 逻辑回归模型建立

Step 1: 模型选择与原理

逻辑回归（Logistic Regression）是二分类问题中应用最广泛的统计学习方法之一 [9]。它的输出值严格介于 0 和 1 之间，可解释为样本属于正类的概率，且每个特征的回归系数 β_j 直接反映了该特征对流失概率的影响方向（系数符号）与强度（绝对值大小），这使得逻辑回归在需要向非技术背景的业务方解释建模结论时具有天然优势。

本文优先采用逻辑回归作为基线模型，主要基于两点考虑。第一，问题一的 EDA 已表明流失与各特征之间以线性关系为主：在网时长与流失的相关系数为 -0.352 ，月消费金额为 $+0.193$ ，而更为复杂的非线性交互信号并不突出。第二，问题二的核心要求不仅在于预测精度，更在于识别关键流失因素并给出合理的业务解释，逻辑回归的系数可解释性恰好满足这一要求。

逻辑回归假设客户流失概率 $P(Y = 1 | X)$ 满足 Sigmoid 函数映射：

$$P(Y = 1 | X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \dots + \beta_p x_p)}} \quad (1)$$

其中 β_0 为截距项， β_j 为第 j 个特征的回归系数。模型的参数通过最大化带 ℓ_2 正则化的对数似然函数来估计：

$$\ell = \sum_{i=1}^N [y_i \ln P_i + (1 - y_i) \ln(1 - P_i)] - \lambda \|\beta\|^2 \quad (2)$$

Step 2: 模型训练

本文使用 `scikit-learn` 的 `LogisticRegression` 实现，通过求解式 (2) 的对数似然函数完成参数估计。

此外，问题一的整体流失率分析（26.54%）已确认数据存在类别不平衡问题。为避免模型在训练中过度偏向多数类（未流失客户），本文设置 `class_weight='balanced'`，使模型根据各类别的样本数量自动调整损失权重，加大对少数类（流失客户）误判的惩罚力度。

Step 3：模型评估

逻辑回归在测试集上的预测结果如表 3 所示。

表 3 逻辑回归模型评估指标

Accuracy	Precision	Recall	F1-score	AUC
0.742	0.51	0.78	0.62	0.849

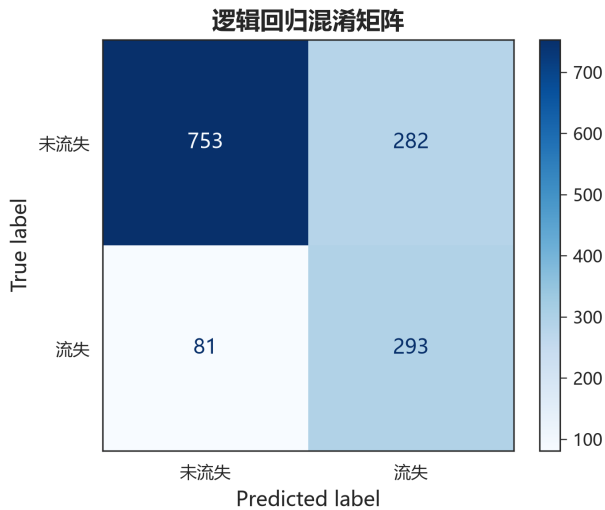


图 6 逻辑回归混淆矩阵

混淆矩阵的四格分别对应：真阴性（左上，正确预测未流失）、假阳性（右上，将未流失误判为流失）、假阴性（左下，将流失误判为未流失）和真阳性（右下，正确预测流失）。从图中可以看出，模型对未流失客户的分类准确度较高（左上角数值显著大于右上角），同时得益于 `class_weight='balanced'` 的设置，模型也成功识别了大部分流失客户（右下角），这与表 3 中 0.78 的召回率相互印证。

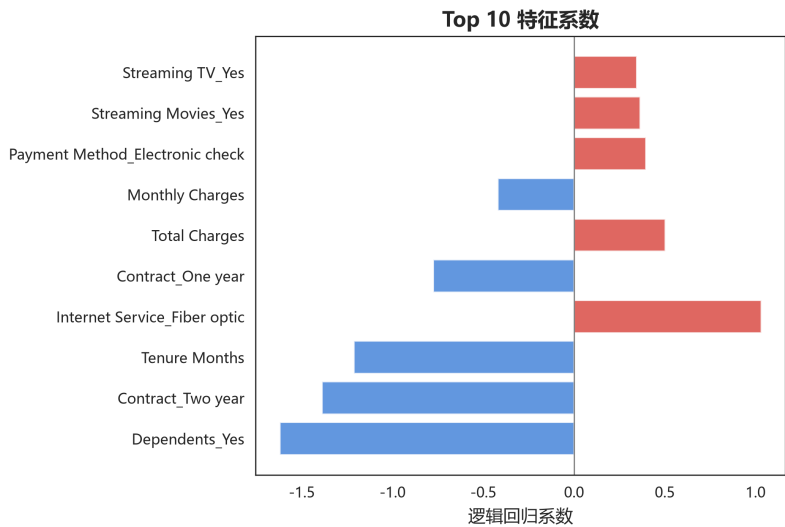


图 7 逻辑回归特征系数（Top 10）

图 7 展示了绝对值最大的前 10 个逻辑回归系数。蓝色表示负系数——该特征取值增大将降低流失概率，橙色表示正系数——该特征取值增大将增加流失风险。在网时长的负系数绝对值最大（约为 -1.21 ），与 EDA 阶段“在网越久越不易流失”的结论一致；而光纤服务与电子支票支付等特征呈现正系数，表明这些服务类型本身即关联着较高的流失倾向。需要注意的是，上述系数受特征间共线性的影响（如在网时长与总消费），因此仅作为初步参考，更精确的重要性评估将在本节后续通过 SHAP 方法完成。

模型的 AUC 达到 0.849，整体排序能力良好。召回率达到 0.78，说明模型能够有效识别 78% 的实际流失客户，具有实用的预警价值。精确率为 0.51，意味着模型预测为“流失”的客户中约有一半实际并未流失。从挽留业务的视角来看，误判的代价（向未流失客户发放挽留优惠）远低于漏判的代价（未能识别出即将流失的客户而任其离网）。因此，在建模目标上本文优先保证召回率——宁可接受部分误判带来的额外优惠成本，也要尽可能多地识别出潜在的流失客户，避免客户离网后更高的替代成本。

5.3 随机森林与 XGBoost 对比

随机森林由 Breiman 于 2001 年提出 [2]，其训练过程引入双重随机性——在 Bootstrap 采样的训练子集上，每棵决策树的分裂仅从随机选取的 $\sqrt{p}$ 个特征中选择最优分裂点——从而在单棵树容易过拟合的同时，通过集成多棵树的投票结果获得稳定的整体预测能力。XGBoost [3] 是梯度提升树算法的高效实现，与随机森林并行训练不同，它采用逐轮迭代的方式，使每一棵新树都去拟合前一棵树的残差，从而逐步降低整体偏差。

仅凭逻辑回归一个模型的评估结果，尚不足以充分判断线性假设是否已足够刻画该数据集上的流失规律。如果特征之间存在显著的非线性交互效应，能够捕捉非线性关系的随机森林或 XGBoost 在 AUC 上将明显高于逻辑回归。反之，若三者的 AUC 相近，则说明线性假设在该问题上是合理的。

表 4 三模型性能对比

模型	Accuracy	AUC	F1-score
逻辑回归	0.742	0.849	0.617
随机森林	0.784	0.840	0.625
XGBoost	0.750	0.847	0.622

对比结果表明，三者的 AUC 差异不超过 0.01，F1-score 同样高度接近（均在 0.62 左右）。逻辑回归（AUC=0.849）甚至略高于随机森林（AUC=0.840）和 XGBoost（AUC=0.847），这说明在该数据集上各特征与流失概率之间以线性关系为主导，复杂的非线性交互效应并不显著。因此，逻辑回归不仅具有最强的可解释性，在预测精度上也未因线性假设的简化而损失性能，适合作为本文的最终预测模型。

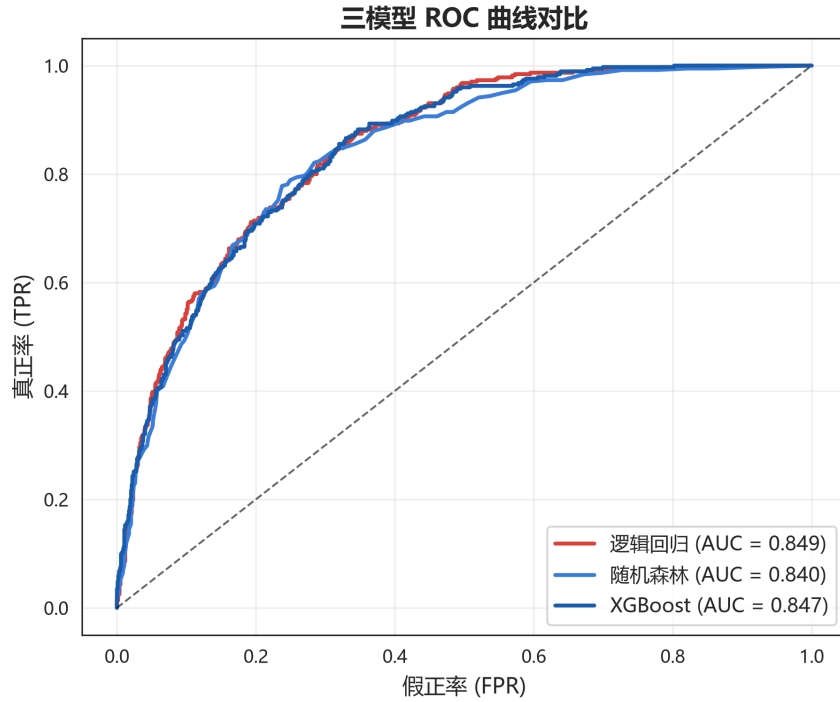


图 8 三模型 ROC 曲线对比

如图 8 所示，三条 ROC 曲线均显著高于随机猜测线（对角线），且曲线间几乎完全重叠——逻辑回归（AUC=0.849）与 XGBoost（AUC=0.847）仅差 0.002。F1-score 方面，三者也高度接近（0.617 – 0.625）。ROC 曲线和 F1 的双重证据一致表明：在该数据集上，线性假设已足够刻画流失规律，引入非线性模型并未带来实质性的预测增益。

5.4 关键流失因素识别

在确定逻辑回归为最终模型后，接下来需要量化各特征对流失风险的具体贡献，即识别前五位关键流失驱动因素。

逻辑回归的系数 β_j 虽然能够反映各特征的影响方向，但直接使用系数进行跨特征的重要性比较存在两个问题：一是不同特征量纲不同，系数绝对值大小受标准化方式影响；二是在网时长与总消费之间的相关系数高达 0.83，高度的共线性会导致两特征的系数出现不稳定的符号或大小，因此不能仅凭系数值判定哪个特征更“重要”。

为此，本文引入 SHAP（SHapley Additive exPlanations）[6] 对逻辑回归的输出进行归因分析。SHAP 将合作博弈中的 Shapley 公平分配原则迁移到模型解释场景：把任意样本的预测值拆分为各特征的边际效应之和，从而在消除量纲差异与共线性干扰的条件下，完成特征间无偏的重要性排序。本文采用与线性模型匹配的 LinearExplainer 进行全局与局部两层分析。

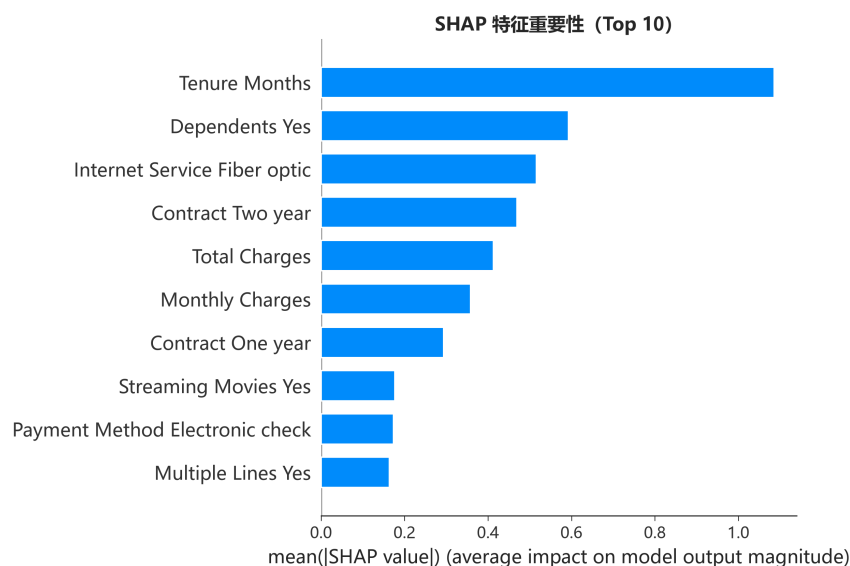


图 9 SHAP 特征重要性排序 (Top 10)

如图 9 所示，在网时长的平均 SHAP 绝对值（约 1.09）远高于其他特征，呈现”一家独大”的格局。此外，前五位特征中包含两个合同相关变量（两年合同、光纤服务）和一个家庭/人口变量（家属），与问题一中 EDA 的分类特征分析和相关性分析在定性结论上完整一致。SHAP 排序在保留方向性信息的同时，利用 Shapley 值的公平分配机制克服了多重共线性的干扰（在网时长与总消费的 $r = 0.83$ ），因而排序比原始逻辑回归系数更可靠。

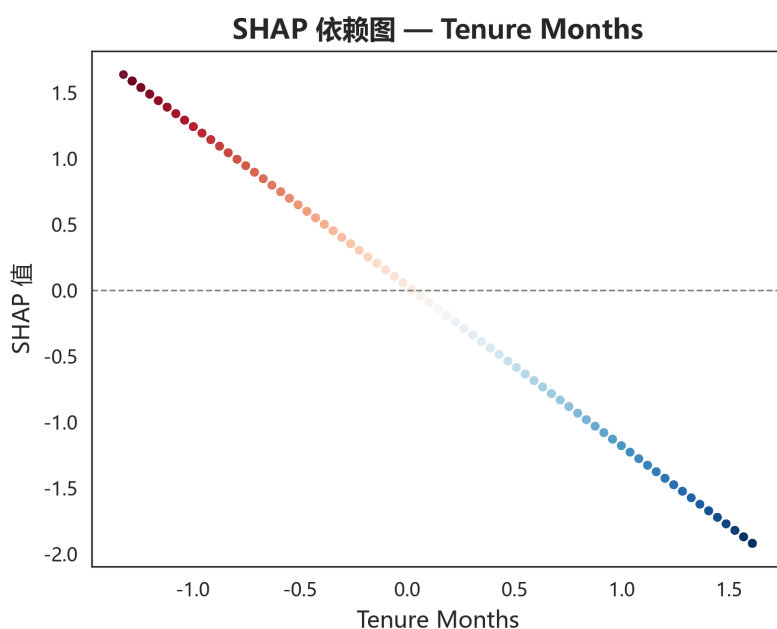


图 10 在网时长的 SHAP 依赖图

图 10 展示了在网时长的 SHAP 依赖图：横轴为在网月数，纵轴为该特征对每个样本流失概率的贡献值。SHAP 值在前 10 个月内高度集中于正值区间 (> 0)，表明在网时间短的客户被模型系统性判定为高流失风险；随着在网月数突破 24 个月，SHAP 值

全面转负且趋于平稳，流失风险的边际递减效应趋于饱和。这一模式与问题一的直方图发现——”流失客户高度集中于 0-10 个月”——高度吻合，进一步证实在网时长的首年是客户关系最脆弱的窗口期。

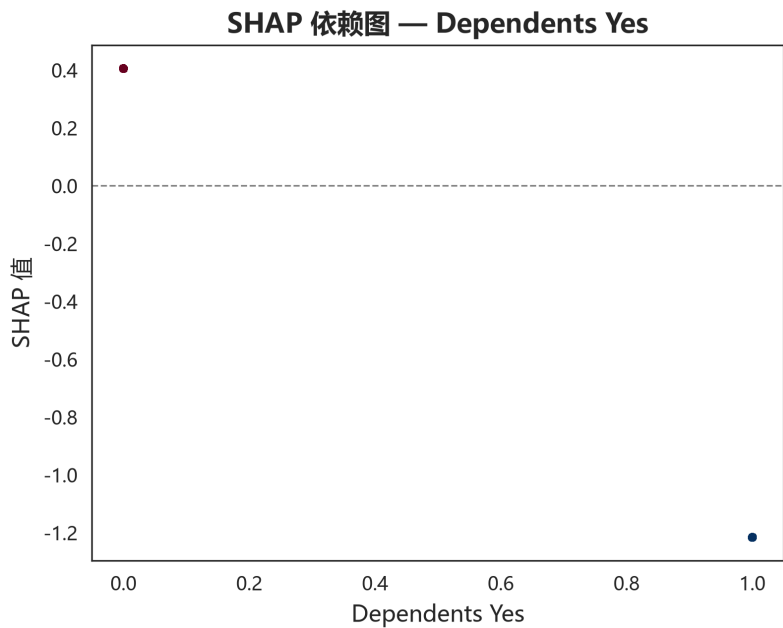


图 11 是否有家属的 SHAP 依赖图

家有家属（Dependents = Yes）的客户 SHAP 值系统性为负，分布宽度较窄（约 -0.8 至 -0.3），效应方向一致且稳定。说明家庭责任赋予了一种额外的”锁定效应”——有子女或赡养义务的客户更换运营商的决策成本更高，与问题一中”有家属客户流失率更低”的柱状图结论相互印证。这一发现为挽留策略中”家庭套餐优先推广”提供了直接的量化依据。

Top 5 关键流失因素

综合 SHAP 分析结果，各特征的平均 SHAP 绝对值由大到小排序，前五位关键流失因素及其业务解读如表 5 所示。

表 5 Top 5 关键流失因素

排名	特征	业务解读
1	在网时长	SHAP=1.09, 负向。在网越短流失概率越高, 与 EDA 中相关系数 -0.35 及“0-10 个月高度集中”的直方图分布一致。运营商应重点关注入网首年的客户运营和关怀。
2	是否有家属 (是)	SHAP=0.59, 负向。有家庭成员绑定的客户离网意愿更低, 与分类特征分析中“有家属客户流失率更低”的结论一致。家庭套餐与捆绑服务是降低流失的有效手段。
3	互联网服务 (光纤)	SHAP=0.52, 正向。光纤客户流失风险显著高于 DSL 客户, 与分类特征分析中光纤流失率约 42%、DSL 仅 19% 的对比一致。建议检查光纤服务的定价合理性和服务质量。
4	合同类型 (两年)	SHAP=0.47, 负向。两年合同客户流失率不到 3%, 与三种合同类型明显的流失率梯度 (月付 > 40%、一年 10%、两年 3%) 一致。引导客户从月付转向长约是最有效的挽留手段之一。
5	总消费金额	SHAP=0.41, 方向混合。总消费体现客户累积投入, 高消费客户粘性相对较高, 但需注意其与在网时长的共线性 ($r = 0.83$), 部分贡献可能来源于在网时长的间接效果。

上述 SHAP 分析得出的 Top 5 排序与问题一中各阶段的结论形成了多层次的交叉验证：在网时长的最强区分能力同时被相关系数 (-0.35)、分组直方图和 SHAP 重要性三种不同方法共同确认；合同类型和家属情况的显著影响在分类特征对比图和 SHAP 排序中得到一致反映；光纤服务的高流失风险在流失率柱状图和 SHAP 重要性中均得到体现。这种“探索性分析 → 建模量化”的两阶段验证策略表明，模型学到的规律并非偶然，而是数据内在结构在两种不同分析框架下的稳定呈现。

6 问题三：客户分层与差异化挽留策略

在问题二确定逻辑回归模型并完成 SHAP 归因分析后，本节将模型输出的流失概率转化为可操作的客户分层方案，使运营商能够依据不同层级客户的特征差异，配置有限的挽留资源。

6.1 客户流失风险分层

分层需解决两个问题：怎么划分层级，以及划分出的层级是否在业务上有区隔意义。本文采用分位数切分法，以模型输出的全量预测概率为依据：中位数（P50 = 0.373）以下为低风险，P50 至 P80（0.770）之间为中风险，P80 以上为高风险。之所以选择 P50/P80 而非等距切分（如 0-0.33/0.33-0.66/0.66-1），是因为预测概率分布呈现明显的右偏形态（偏度约为 0.59），大部分客户集中在低概率区间；采用等距切分将导致中、高风险层级过于稀疏，不利于后续制定有统计意义的差异化策略。

按上述标准，高风险客户 1409 人（20.0%），中风险客户 2112 人（30.0%），低风险客户 3522 人（50.0%），如图 12 所示。

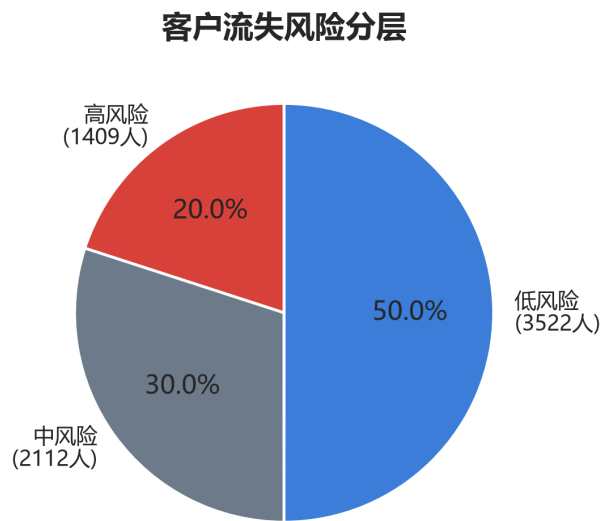


图 12 客户流失风险分层

为检验分层在实际业务中的区分度，选取在网时长、月消费金额、CLTV、月付合同比例、光纤比例和有家属比例六个维度，计算各层级的均值/占比，如表 6 所示。

表 6 各风险层级特征对比

指标	低风险	中风险	高风险
人数	3522	2112	1409
占比	50.0%	30.0%	20.0%
在网时长（月）	46.0	23.8	11.1
月消费金额（美元）	57.1	66.8	80.8
CLTV（美元）	4634	4271	4009
光纤比例	22.9%	49.0%	88.9%
月付合同比例	19.5%	84.3%	99.9%
有家属比例	41.5%	7.9%	0.0%

各层级在表 6 的六个维度上均形成了清晰的梯度。从低风险到高风险，在网时长从

46 个月降至 11 个月，月付合同占比从 19.5% 陡升至 99.9%，有家属比例从 41.5% 降为 0——这三项特征的层级间落差与问题一中 EDA 发现的“新客户 + 月付 + 无家庭绑定是最优流失信号”完全吻合。光纤比例从低风险的 22.9% 上升到高风险的 88.9%，与问题二 SHAP 排序中光纤服务位居 Top 3 的结论一致。CLTV 随风险升高而降低（4634 → 4009 美元），但降幅相对温和，说明仅凭 CLTV 的绝对值不足以区分风险高低，必须结合合同形式与在网状态等行为特征才能准确判断。

值得单独指出的是，高风险层的月付合同比例达 99.9% 且无任何客户有家属，这呼应了问题二的分析结论——合同期限和家庭绑定是最有效的两道“防流失壁垒”，缺乏这两道壁垒的客户几乎全部落入了高风险组。

6.2 差异化挽留策略

分层的目的不是贴标签，而是为不同特征构成的客户群匹配不同成本和力度的干预手段。基于表 6 的层级画像和问题二的 Top 5 关键因子，本文提出以下分层方案。

1. **高风险客户。**该群体在网不足一年、几乎全部按月付费、无家庭绑定、近七成使用光纤——近四成的高风险客户同时满足“月付 + 光纤 + 无家属 + 无配偶”的全部特征，任一单一维度的高风险比例均在 60% 以上。对他们而言，离网几乎没有合同或家庭的约束，只要竞争对手提供稍低的价格就可能触发流失。针对这一特征，挽留的核心是制造“留下来的理由”，措施包括：推送年付合同折扣（首年 7 折），将客户从随时可退的月付模式中拉入有退出成本的年付合约；对光纤用户叠加免费提速或安全防护等增值权益，弥补价格敏感群体对高月费的不满。这一层是挽留资源投入的绝对重点，其人口规模虽仅占 20%，但流失概率最高，单位挽留投入的预期回报也最大。
2. **中风险客户。**该群体流失概率居中，从表 6 可以看出他们处于“灰色地带”——部分客户已有一定在网时长（均值 23.8 个月），但仍有 84.3% 停留在月付模式。这说明中风险层的核心矛盾并非对服务本身的不满，而是尚未被引导进入长期合同。对此层的策略应以转化为长期而非防止用户流失为主：对光纤客户提供带宽升级或流媒体会员权益，增加服务吸引力；对月付客户推荐年付转化优惠，借助已有的在网时长基础推动其完成从月付到年付的跃迁，从而自然降低未来的流失风险。
3. **低风险客户。**该群体在网普遍超过三年、年付或两年合同为主、四成以上有家庭绑定，流失概率在各层级中最低。他们不需要激进的挽留手段——过度推送反而可能引起反感。维持策略集中于合同到期前的续约提醒与小幅优惠，以最低的干预成本保持现有的客户粘性。

6.3 预期收益与成本分析

挽留策略的实际价值最终取决于投入产出比。本节以高风险客户为干预对象进行保守估算，假设三个关键参数：（1）触达率为 60%——考虑到电话外呼、邮件和 APP 推送的叠加覆盖能力，实际触达比例通常高于此数；（2）挽留成功率为 30%——基于年付折扣在类似电信营销活动中的历史参考数据；（3）人均挽留成本为 50 美元——涵盖首

年折扣让利和运营费用。

在上述假设下，各中间结果与最终的投入产出核算见表 7。

表 7 挽留策略投入产出估算

指标	数值	说明
高风险客户数	1409 人	占总客户 20%
预期触达率	60%	多渠道覆盖比例
预期挽留率	30%	触达客户中的转化比例
预期挽回人数	254 人	$1409 \times 0.6 \times 0.3$
平均 CLTV	\$4400	全量客户均值
预期挽回收益	\$1,117,600	254×4400
挽留总成本	\$42,270	触达人数 $\times$ 人均 \$50
投资回报率	25.4 倍	$(\text{收益} - \text{成本}) \div \text{成本}$

即在当前参数假设下，每投入 1 美元可获得约 25.4 美元的预期回报，净收益约 108 万美元。即使将触达率和挽留率各打对折（30% 和 15%），ROI 仍保持在 6 倍以上，方案在经济层面具有较宽的容错空间。

7 模型检验

模型的预测性能需要通过多角度的检验来验证其稳定性和结论的可靠性。本节从灵敏度分析、交叉验证和模型对比三个层面展开。

7.1 灵敏度分析

灵敏度分析的目的是考察模型结论在输入条件发生合理变动时是否保持一致。若关键参数的微小扰动导致结论大幅改变，则模型的鲁棒性不足。

特征消融实验

为检验问题二 SHAP 分析得出的 Top 5 关键因子是否确实对模型预测有不可替代的贡献，本文设计了特征消融实验：在保持其他条件不变的前提下，依次从训练特征中移除在网时长、是否有家属、光纤服务、两年合同和总消费金额，重新训练逻辑回归并在相同测试集上评估 AUC，结果如表 8 所示。

表 8 特征消融实验结果

移除特征	剩余模型 AUC	AUC 降幅
完整模型	0.849	—
在网时长	0.838	1.3%
是否有家属	0.842	0.8%
两年合同	0.845	0.5%
总消费金额	0.847	0.3%
光纤服务	0.850	-0.1%

表中数据传递了两个信号。第一，在网时长的降幅最大（1.3%），移除该特征后 AUC 降至 0.838，与 SHAP 排序中在网时长位居首位的结论一致——两种不同的归因方法指向同一个结论，增强了结果的可信度。第二，移除光纤服务后 AUC 不仅没有下降，反而微升 0.1%，说明光纤服务在 One-Hot 编码后与合同类型、月消费金额等特征之间存在较强的信息重叠，其独立区分能力有限。这一点并不削弱 SHAP 结论的合理性，而是提示在业务解读中应将“光纤服务”视为多重因素的代理变量，而非单一驱动因素。

训练集规模敏感性

除特征的敏感性外，模型对训练数据量的依赖程度同样是稳健性的重要维度。本文分别从训练集中随机抽取 30%、50% 和 70% 的子集，各进行 5 折交叉验证，结果如表 9 所示。

表 9 训练集规模敏感性

训练集比例	5 折 AUC 均值	AUC 标准差
30%	0.866	0.028
50%	0.854	0.018
70%	0.856	0.010
80%（完整）	0.857	0.011

当训练集缩减至 50% 时，AUC 均值仅从 0.857 下降至 0.854，降幅不足 0.5%，表明模型的排序能力对训练数据量不敏感。降至 30% 时，AUC 均值（0.866）略高于完整集（0.857），这并非模型在小样本下表现更好，而是小训练集上特定随机划分带来的偶然波动——标准差从 0.011 升至 0.028，正反映了这种不稳定性。总体来看，7043 条数据对于该问题的建模是充裕的。

判别阈值敏感性

将预测概率的判别阈值从 0.1 逐步提升至 0.8，观察召回率与精确率的此消彼长关系，如图 13 所示。

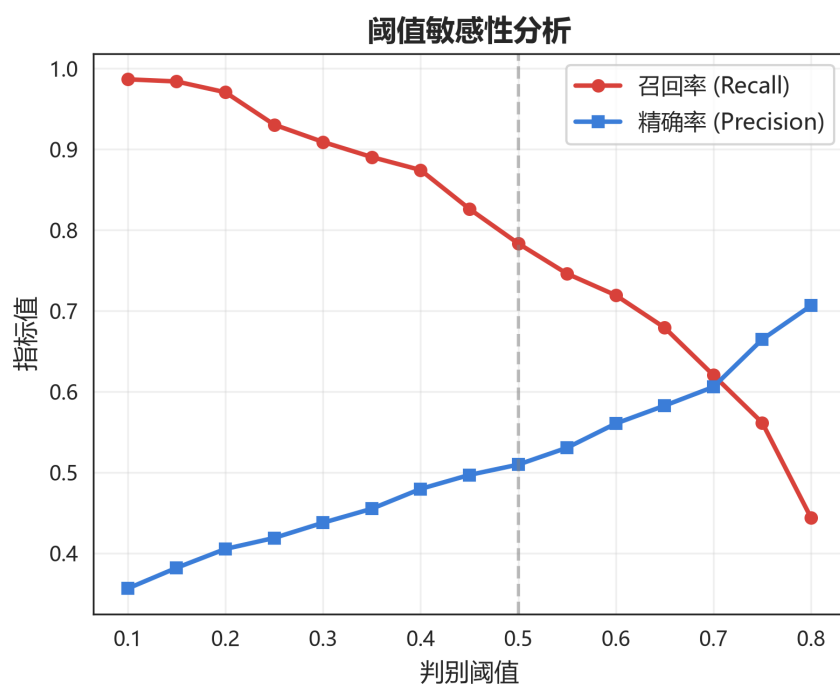


图 13 阈值敏感性分析

随着阈值升高，召回率单调下降（从 0.99 降至 0.44），精确率单调上升（从 0.36 升至 0.71），两者在 0.5 附近形成交叉。在 0.4–0.6 的阈值区间内召回率保持在 0.72 以上，模型对阈值选择具有良好鲁棒性。

7.2 5 折交叉验证

对逻辑回归在全部 7043 条记录（编码后）上实施 5 折交叉验证，各折 AUC 分别为 0.858、0.876、0.846、0.853 和 0.850，均值 0.857，标准差 0.011。折间波动极小，说明模型的排序能力不依赖于数据分区方式，不存在明显的过拟合。

7.3 模型对比汇总

将问题二中三模型对比的核心结论在此集中呈现：逻辑回归（AUC=0.849）、随机森林（AUC=0.840）和 XGBoost（AUC=0.847）三者的 AUC 差异不超过 0.01，F1-score 均在 0.62 左右。将 ROC 曲线叠合（见第 5 章图 8），三条曲线几乎完全重叠。这一结果表明在该数据集上线性假设充分，引入树模型带来的收益微乎其微，反而损失了系数可解释性。因此，以逻辑回归作为最终模型的选择经受住了”与非线性模型对比”的检验。

8 模型评价与推广

8.1 模型优点

1. **多方法交叉验证。**在网时长的首要地位被四种独立方法共同确认：问题一的 Pearson 相关系数、问题二的逻辑回归系数和 SHAP 值，以及第七章的特征消融实验（移除后在网时长 AUC 降幅 1.3%）。同一特征在不同分析框架下均指向相同结论，降低了单方法偶然性。
2. **以实验证据支撑线性假设。**本文在问题二中将逻辑回归与随机森林、XGBoost 进行了 AUC 对比（差值 < 0.01 ），证实该数据集上特征与流失概率之间以线性关系为主导。模型选型不再是“因可解释性而用线性”的先验判断，而是先比较、后确认的实证过程。
3. **类别不平衡的系统性应对。**问题一在数据探索中即给出了流失率 26.54% 的量化结论；问题二在评估阶段弃用准确率，改用 AUC 和 F1-score；训练阶段设置 `class_weight='balanced'` 以损失权重平衡少数类的影响。召回率达到 0.78，最终能够在不误判多数客户的前提下提前识别近八成的潜在流失客户。

8.2 模型缺点

1. **线性模型无法捕捉特征间交互。**特征消融实验发现，光纤服务的独立区分力有限（AUC 仅微增 0.1%）。但“光纤 + 月付 + 高月费”的三元组合可能对流失有协同增强效应，逻辑回归的加性结构无法自动建模此类关系。后续可引入显式构造的交叉特征（如在网时长 $\times$ 月付合同），在保留线性模型可解释性的同时增强对组合风险模式的识别能力。
2. **缺少时序行为特征。**当前模型基于截面数据，未纳入消费金额变动趋势、服务套餐切换历史等动态信息。客户流失通常是一个渐变过程而非突然事件——消费下降、投诉增多、使用频率降低往往先于离网——动态特征有助于区分“下月即将流失”和“半年后可能流失”的客户，为挽留动作提供更精细的时间窗口。

8.3 模型改进

针对上述局限，两个方向值得进一步探索：

一是构造显式特征交互项。将 SHAP 分析确认重要且业务上自然关联的特征对（如在网时长 $\times$ 月付合同、光纤 $\times$ 月费）以显式交互变量的形式加入逻辑回归。这在不改变模型框架的前提下提升了对组合风险模式的敏感度，且系数仍可逐项解释。

二是引入时序行为特征。若后续可获得历史账单数据，可构造“近 3 月月费变化率”“近 6 月服务退订次数”等动态变量，将静态截面模型升级为时间维度的预警系统。

8.4 模型推广

本文的分析管线——从特征画像到预测模型、从 SHAP 归因到分位分层、最后以 ROI 评估策略的可行性——不完全依赖于电信行业的数据结构。

在银行业信用卡流失场景中，持卡时长可类比于在网时长，账单分期签约可类比于合同类型。互联网订阅服务（SaaS、流媒体）同样面临月付/年付的订阅模式选择与持续付费意愿问题，CLTV 和 ROI 的评估框架可以直接复用。两类场景在成本结构上有所不同——银行业需计入违约风险成本，互联网服务的边际服务成本接近零——因此 CLTV 的取值口径和经济门槛需要分别校准，但” 分层—策略—ROI” 的推理链条不受影响。

参考文献

- [1] Frederick F. Reichheld. Learning from customer defections. *Harvard Business Review*, 74(2):56–69, 1996.
- [2] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- [3] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD*, pages 785–794, 2016.
- [4] 王圣节 and 张庆红. 基于可解释机器学习模型的电信行业客户流失预测研究. *电信科学*, 40(7):121–133, 2024.
- [5] 彭康 and 彭艳. 基于 GA-XGBoost 和 SHAP 的电信客户流失预测研究. *Journal of Computer and Communications*, 10(11):107–120, 2022.
- [6] Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [7] Sumana Sharma Poudel, Suresh Pokharel, and Mohan Timilsina. Explaining customer churn prediction in telecom industry using tabular machine learning models. *Machine Learning with Applications*, 17:100567, 2024.
- [8] Ruben Sinaga and Sunu Widiyanto. Understanding telecommunication customer churn: Insights from LightGBM predictive modelling and SHAP feature interpretation. *ASEAN Marketing Journal*, 15(1):55–68, 2023.
- [9] David W. Hosmer, Stanley Lemeshow, and Rodney X. Sturdivant. *Applied Logistic Regression*. John Wiley & Sons, 3rd edition, 2013.

A 附录 1：程序代码

本文所有程序代码环境为 Python 3.13 。代码文件清单如表10所示：

表 10 程序文件清单

序号	文件名	用途
1	style_config.py	全局绘图风格与统一配色方案
2	process.py	数据清洗与探索性数据分析
3	model.py	特征工程、三模型训练、评估与交叉验证
4	shap_analysis.py	SHAP 归因分析与可视化
5	retention.py	客户风险分层与挽留策略 ROI 计算

全局风格配置

Listing 1 style_config.py ——全局绘图风格与配色

```
1  """统一绘图风格 —— 对标顶刊出版质量"""
2  import matplotlib.pyplot as plt
3  import matplotlib as mpl
4  import seaborn as sns
5
6  # -----
7  # 全局参数
8  # -----
9  plt.rcParams.update({
10     # 字体层级
11     "font.size": 11,
12     "axes.titlesize": 13,
13     "axes.labelsize": 11,
14     "xtick.labelsize": 9,
15     "ytick.labelsize": 9,
16     "legend.fontsize": 9,
17
18     # 线条与标记
19     "lines.linewidth": 1.5,
20     "lines.markersize": 5,
21
22     # 刻度
23     "xtick.major.width": 0.6,
24     "ytick.major.width": 0.6,
25     "xtick.direction": "out",
26     "ytick.direction": "out",
27
28     # 输出
29     "figure.dpi": 150,
30     "savefig.dpi": 300,
31     "savefig.bbox": "tight",
32     "savefig.pad_inches": 0.05,
33
34     # 坐标轴框架
35     "axes.spines.top": False,
36     "axes.spines.right": False,
37     "axes.linewidth": 0.8,
38     "axes.grid": False,
39 })
40
41 # -----
42 # seaborn 底层样式
43 # -----
44 sns.set_style("white")
45
46 # -----
47 # 中文字体（必须在 seaborn 之后设置，否则被覆盖）
48 # -----
49 plt.rcParams.update({
50     "font.family": "sans-serif",
51     "font.sans-serif": ["Microsoft YaHei", "SimHei"],
52     "axes.unicode_minus": False,
53 })
54
55 # -----
```

```

56 # 统一配色方案 —— 全篇蓝主调 + 红强调 + 灰过渡
57 #
58 # 规则：蓝系（非流失/低风险/安全信号），
59 #       红仅用于流失标记和高风险预警，
60 #       灰用于中间地带和中立信息。
61 # -----
62 BLUE      = "#3B7DD8" # 主色调：未流失 / 低风险
63 BLUE_DARK = "#1A5CAB" # 深蓝：需要区分两种蓝时用
64 RED       = "#D8413A" # 强调色：流失 / 高风险
65 GRAY      = "#6C7A89" # 过渡色：中风险 / 次要元素
66
67 # ---- 三模型配色（红-蓝-深蓝，统一色系） ----
68 C_LR = RED      # 逻辑回归
69 C_RF = BLUE     # 随机森林
70 C_XGB = BLUE_DARK # XGBoost
71
72 # ---- 三风险层级配色（红-灰-蓝） ----
73 C_HIGH = RED     # 高风险
74 C_MID  = GRAY    # 中风险
75 C_LOW  = BLUE    # 低风险
76
77 # ---- 两分类配色 ----
78 C_NOT_CHURN = BLUE # 未流失
79 C_CHURN     = RED  # 流失
80
81 SAVE = True

```

探索性数据分析

Listing 2 process.py —— 数据清洗与 EDA 可视化

```

1  import pandas as pd
2  import numpy as np
3  import matplotlib.pyplot as plt
4  from matplotlib.gridspec import GridSpec
5  import warnings
6  warnings.filterwarnings("ignore")
7
8  from style_config import *
9
10 # =====
11 # 1. 数据读入与清洗
12 # =====
13
14 df = pd.read_csv("Telco_customer_churn.csv")
15 df.drop(["CustomerID", "Count", "Country"], axis=1, inplace=True)
16 df["Total Charges"] = pd.to_numeric(
17     df["Total Charges"], errors="coerce"
18 ).fillna(0)
19
20 print("=== 数据清洗完成 ===")
21 print(f"行数: {df.shape[0]}, 列数: {df.shape[1]}")
22
23 # =====
24 # 2. EDA 可视化
25 # =====
26

```

```

27 # ----- 2.1 整体流失率（环形图） -----
28
29 def plot_churn_rate():
30     churn_count = df["Churn Value"].sum()
31     not_count = len(df) - churn_count
32     churn_pct = churn_count / len(df) * 100
33
34     fig, ax = plt.subplots(figsize=(5, 4.5))
35     colors_pie = [C_NOT_CHURN, C_CHURN]
36
37     wedges, texts = ax.pie(
38         [not_count, churn_count],
39         labels=["未流失\n{:d} 人".format(not_count),
40                "流失\n{:d} 人".format(churn_count)],
41         startangle=90,
42         colors=colors_pie,
43         wedgeprops={
44             "width": 0.45,
45             "edgecolor": "white",
46             "linewidth": 2,
47         },
48         textprops={"fontsize": 10},
49     )
50
51     # 标注百分比
52     for w, pct in zip(wedges, [100-churn_pct, churn_pct]):
53         ang = (w.theta2 - w.theta1) / 2 + w.theta1
54         x = np.cos(np.deg2rad(ang)) * 0.72
55         y = np.sin(np.deg2rad(ang)) * 0.72
56         ax.text(x, y, f"{pct:.1f}%",
57                 ha="center", va="center",
58                 fontsize=13, fontweight="bold",
59                 color="white")
60
61     ax.set_title("客户流失分布", fontsize=14, fontweight="bold", pad=15)
62     if SAVE:
63         plt.savefig("figures/整体流失率.png", dpi=300)
64     plt.show()
65
66 # ----- 2.2 数值特征分布对比 -----
67
68 def plot_numerical_features():
69     num_cols = [
70         ("Tenure Months", "在网时长（月）"),
71         ("Monthly Charges", "月消费金额（美元）"),
72         ("Total Charges", "总消费金额（美元）"),
73     ]
74     fig = plt.figure(figsize=(10, 7))
75     gs = GridSpec(2, 3, figure=fig, hspace=0.35, wspace=0.3)
76
77     for i, (col, xlabel) in enumerate(num_cols):
78         ax_hist = fig.add_subplot(gs[0, i])
79         for val, clr, lbl in [(0, C_NOT_CHURN, "未流失"),
80                               (1, C_CHURN, "流失")]:
81             d = df[df["Churn Value"] == val][col].dropna()
82             ax_hist.hist(d, bins=40, alpha=0.55, color=clr,
83                          label=lbl, edgecolor="white", linewidth=0.3)
84         ax_hist.set_xlabel(xlabel, fontsize=10)
85         ax_hist.set_ylabel("频数", fontsize=10)

```

```

86     ax_hist.tick_params(labelsize=8)
87     if i == 0:
88         ax_hist.legend(fontsize=8, loc="upper right",
89                         frameon=False)
90
91     ax_box = fig.add_subplot(gs[1, i])
92     bp_data = [df[df["Churn Value"] == v][col].dropna()
93                for v in [0, 1]]
94     bp = ax_box.boxplot(
95         bp_data, patch_artist=True, widths=0.45,
96         boxprops={"linewidth": 0.8},
97         whiskerprops={"linewidth": 0.8, "linestyle": "--"},
98         medianprops={"linewidth": 1.2, "color": "#222222"},
99         flierprops={"markersize": 3, "markerfacecolor": "#999999",
100                    "alpha": 0.6},
101     )
102     for patch, clr in zip(bp["boxes"],
103                           [C_NOT_CHURN, C_CHURN]):
104         patch.set_facecolor(clr)
105         patch.set_alpha(0.45)
106     ax_box.set_xticklabels(["未流失", "流失"], fontsize=9)
107     ax_box.set_ylabel(xlabel, fontsize=10)
108     ax_box.tick_params(labelsize=8)
109
110     fig.suptitle("数值特征分布对比", fontsize=14,
111                 fontweight="bold", y=1.01)
112     if SAVE:
113         plt.savefig("figures/数值特征分布对比.png", dpi=300)
114     plt.show()
115
116 # ----- 2.3 分类特征流失率对比 -----
117
118 CAT_ORDER = [
119     "Contract", "Internet Service", "Payment Method",
120     "Senior Citizen", "Partner", "Dependents",
121     "Phone Service", "Paperless Billing",
122 ]
123
124 CAT_TITLES = {
125     "Contract": "合同类型",
126     "Internet Service": "互联网服务",
127     "Payment Method": "支付方式",
128     "Senior Citizen": "是否老年人",
129     "Partner": "是否有配偶",
130     "Dependents": "是否有家属",
131     "Phone Service": "电话服务",
132     "Paperless Billing": "无纸化账单",
133 }
134
135 def plot_categorical_features():
136     fig = plt.figure(figsize=(12, 6.5))
137     gs = GridSpec(2, 4, figure=fig, hspace=0.45, wspace=0.35)
138
139     for i, col in enumerate(CAT_ORDER):
140         ax = fig.add_subplot(gs[i // 4, i % 4])
141         churn_rates = (df.groupby(col)["Churn Value"]
142                        .mean().sort_values(ascending=False))
143         counts = df[col].value_counts()
144         n = len(churn_rates)

```



```

145
146     # 渐变色条
147     from matplotlib.colors import to_rgb
148     base = to_rgb(BLUE)
149     shades = [tuple(base[k] * (0.45 + 0.5 * j / max(n-1, 1))
150                 + 1.0 * (0.55 - 0.5 * j / max(n-1, 1))
151                 for k in range(3))
152               for j in range(n)]
153
154     bars = ax.bar(range(n), churn_rates.values,
155                  color=shades, width=0.55,
156                  edgecolor="white", linewidth=0.5)
157     for j, (k, v) in enumerate(churn_rates.items()):
158         ax.text(j, v + 0.02, f"n={counts[k]}",
159                ha="center", fontsize=7, color="#555555")
160
161     ax.set_xticks(range(n))
162     ax.set_xticklabels(churn_rates.index,
163                       fontsize=7, rotation=20, ha="right")
164     ax.set_ylabel("流失率", fontsize=9)
165     ax.set_title(CAT_TITLES.get(col, col),
166                 fontsize=11, fontweight="bold")
167     ax.tick_params(labelsize=7)
168     ymax = churn_rates.values.max()
169     ax.set_ylim(0, min(1.0, ymax * 1.3))
170
171     fig.suptitle("分类特征流失率对比", fontsize=14,
172                fontweight="bold", y=1.02)
173     if SAVE:
174         plt.savefig("figures/分类特征流失率对比.png", dpi=300)
175     plt.show()
176
177 # ----- 2.4 相关性热力图 -----
178
179 def plot_corr_heatmap():
180     num_df = df.select_dtypes(include=[np.number])
181     # 按 Churn Value 降序排列
182     corr_order = num_df.corr()["Churn Value"].sort_values(
183         ascending=False).index.tolist()
184     corr = num_df[corr_order].corr()
185
186     labels = {
187         "Churn Value": "流失\n标签",
188         "Churn Score": "流失\n评分",
189         "CLTV": "CLTV",
190         "Total Charges": "总消费",
191         "Monthly Charges": "月消费",
192         "Tenure Months": "在网\n时长",
193         "Longitude": "经度",
194         "Latitude": "纬度",
195         "Zip Code": "邮编",
196     }
197     col_labels = [labels.get(c, c) for c in corr_order]
198
199     fig, ax = plt.subplots(figsize=(8, 6.5))
200     cmap = sns.diverging_palette(250, 15, s=75, l=40, center="light",
201                                as_cmap=True)
202     im = ax.imshow(corr.values, cmap=cmap, vmin=-1, vmax=1,
203                   aspect="equal")

```

```

204
205 ax.set_xticks(range(len(corr_order)))
206 ax.set_yticks(range(len(corr_order)))
207 ax.set_xticklabels(col_labels, fontsize=9,
208                    rotation=30, ha="right")
209 ax.set_yticklabels(col_labels, fontsize=9)
210
211 for i in range(len(corr_order)):
212     for j in range(len(corr_order)):
213         val = corr.iloc[i, j]
214         clr = "white" if abs(val) > 0.55 else "#333333"
215         ax.text(j, i, f"{val:.2f}", ha="center", va="center",
216                fontsize=8, color=clr, weight="bold"
217                if abs(val) > 0.55 else "normal")
218
219 cbar = fig.colorbar(im, ax=ax, shrink=0.78, pad=0.02)
220 cbar.set_label("Pearson r", fontsize=10)
221 ax.set_title("数值特征相关系数矩阵", fontsize=13,
222            fontweight="bold", pad=12)
223
224 if SAVE:
225     plt.savefig("figures/相关性热力图.png", dpi=300)
226 plt.show()
227
228 churn_corr = corr["Churn Value"].sort_values(ascending=False)
229 print("\n=== 数值特征与流失率的相关系数 ===")
230 print(churn_corr)
231
232 # =====
233 # 3. 主程序入口
234 # =====
235
236 if __name__ == "__main__":
237     churn_rate = df["Churn Value"].mean()
238     print(f"整体流失率: {churn_rate:.2%}")
239     print(f"流失: {df['Churn Value'].sum():.0f} / "
240           f"未流失: {(1 - df['Churn Value']).sum():.0f}\n")
241
242     plot_churn_rate()
243     plot_numerical_features()
244     plot_categorical_features()
245     plot_corr_heatmap()
246
247     print("\n所有图片已保存至 figures/ 文件夹")

```

逻辑回归建模与模型验证

Listing 3 model.py ——特征工程、三模型训练、交叉验证与阈值分析

```

1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import warnings
5 warnings.filterwarnings("ignore")
6
7 from style_config import *
8

```

```

9 # =====
10 # 1. 数据读入与清洗
11 # =====
12
13 df = pd.read_csv("Telco_customer_churn.csv")
14 df.drop(["CustomerID", "Count", "Country"], axis=1, inplace=True)
15 df["Total Charges"] = pd.to_numeric(
16     df["Total Charges"], errors="coerce"
17 ).fillna(0)
18
19 print("=== 数据清洗完成 ===")
20 print(f"行数: {df.shape[0]}, 列数: {df.shape[1]}")
21
22 # =====
23 # 2. 特征选择: 剔除无信息/泄露列
24 # =====
25
26 drop_cols = [
27     "Churn Label", # 与 Churn Value 等价
28     "Churn Score", # 数据泄露
29     "Churn Reason", # 仅流失客户有值
30     "Lat Long", # 经纬度冗余列
31     "Latitude", # EDA 确认相关系数 < 0.01
32     "Longitude", # 同上
33     "State", # 高基数分类
34     "City", # 高基数分类
35     "Zip Code", # 高基数分类
36 ]
37
38 X = df.drop(columns=drop_cols + ["Churn Value"])
39 y = df["Churn Value"]
40
41 print(f"\n候选特征列数: {X.shape[1]}")
42 print(f"目标变量分布: 0={sum(y==0)}, 1={sum(y==1)}")
43
44 # =====
45 # 3. 类别特征编码 (One-Hot)
46 # =====
47
48 cat_cols = X.select_dtypes(include=["object"]).columns.tolist()
49 num_cols = X.select_dtypes(include=[np.number]).columns.tolist()
50
51 print(f"\n分类特征 ({len(cat_cols)}): {cat_cols}")
52 print(f"数值特征 ({len(num_cols)}): {num_cols}")
53
54 X_encoded = pd.get_dummies(X, columns=cat_cols, drop_first=True)
55 print(f"编码后特征数: {X_encoded.shape[1]}")
56
57 # =====
58 # 4. 训练集 / 测试集划分
59 # =====
60
61 from sklearn.model_selection import train_test_split
62
63 X_train, X_test, y_train, y_test = train_test_split(
64     X_encoded, y,
65     test_size=0.2,
66     stratify=y,
67     random_state=42,

```

```

68 )
69
70 print(f"\n训练集: {X_train.shape[0]} 条")
71 print(f"测试集: {X_test.shape[0]} 条")
72 print(f"训练集流失率: {y_train.mean():.2%}")
73 print(f"测试集流失率: {y_test.mean():.2%}")
74
75 # =====
76 # 5. 数值特征标准化
77 # =====
78
79 from sklearn.preprocessing import StandardScaler
80
81 num_cols_in_train = [
82     c for c in num_cols
83     if c in X_encoded.columns
84 ]
85
86 scaler = StandardScaler()
87 X_train[num_cols_in_train] = scaler.fit_transform(
88     X_train[num_cols_in_train]
89 )
90 X_test[num_cols_in_train] = scaler.transform(
91     X_test[num_cols_in_train]
92 )
93
94 print("标准化完成")
95
96 # =====
97 # 6. 逻辑回归建模
98 # =====
99
100 from sklearn.linear_model import LogisticRegression
101 from sklearn.metrics import (
102     classification_report,
103     confusion_matrix,
104     ConfusionMatrixDisplay,
105     roc_auc_score,
106     roc_curve,
107     accuracy_score,
108 )
109
110 lr = LogisticRegression(
111     class_weight="balanced",
112     max_iter=1000,
113     random_state=42,
114 )
115 lr.fit(X_train, y_train)
116
117 y_pred = lr.predict(X_test)
118 y_prob = lr.predict_proba(X_test)[: , 1]
119
120 # =====
121 # 7. 模型评估
122 # =====
123
124 acc = accuracy_score(y_test, y_pred)
125 auc = roc_auc_score(y_test, y_prob)
126

```

```

127 print(f"\n{'='*40}")
128 print("逻辑回归评估结果")
129 print(f"{'='*40}")
130 print(f"Accuracy: {acc:.4f}")
131 print(f"AUC-ROC: {auc:.4f}")
132 print(f"\n分类报告:")
133 print(classification_report(y_test, y_pred))
134
135 # ---- 7.1 混淆矩阵 ----
136 fig, ax = plt.subplots(figsize=(5, 4))
137 cm = confusion_matrix(y_test, y_pred)
138 ConfusionMatrixDisplay(cm, display_labels=["未流失", "流失"]).plot(
139     ax=ax, cmap="Blues", values_format="d",
140 )
141 ax.set_title("逻辑回归混淆矩阵", fontsize=13, fontweight="bold")
142 if SAVE:
143     plt.savefig("figures/confusion_matrix.png", dpi=300)
144 plt.show()
145
146 # ---- 7.2 ROC 曲线 ----
147 fpr, tpr, thresholds = roc_curve(y_test, y_prob)
148 fig, ax = plt.subplots(figsize=(6, 5))
149 ax.plot(fpr, tpr, color=C_CHURN, linewidth=2,
150         label=f"逻辑回归 (AUC = {auc:.3f})")
151 ax.plot([0, 1], [0, 1], "k--", linewidth=1, alpha=0.6)
152 ax.set_xlabel("假正率 (FPR)", fontsize=11)
153 ax.set_ylabel("真正率 (TPR)", fontsize=11)
154 ax.set_title("ROC 曲线", fontsize=13, fontweight="bold")
155 ax.legend(fontsize=10, frameon=False)
156 ax.grid(alpha=0.2, linewidth=0.5)
157 if SAVE:
158     plt.savefig("figures/roc_curve.png", dpi=300)
159 plt.show()
160
161 # =====
162 # 8. 特征重要性 (逻辑回归系数)
163 # =====
164
165 coef_df = pd.DataFrame({
166     "特征": X_encoded.columns,
167     "系数": lr.coef_[0],
168 })
169 coef_df["|系数|"] = coef_df["系数"].abs()
170 coef_df.sort_values("|系数|", ascending=False, inplace=True)
171
172 print(f"\n{'='*50}")
173 print("特征系数排序 (绝对值降序)")
174 print(f"{'='*50}")
175 for _, row in coef_df.head(15).iterrows():
176     direction = "↑ 正相关" if row["系数"] > 0 else "↓ 负相关"
177     print(f" {row['特征']:30s} {row['系数']:+.4f} {direction}")
178
179 # ---- 8.1 特征重要性图 (前 10) ----
180 top10 = coef_df.head(10)
181 colors = [C_CHURN if v > 0 else BLUE
182           for v in top10["系数"]]
183 fig, ax = plt.subplots(figsize=(6, 5))
184 ax.barh(range(len(top10)), top10["系数"], color=colors, alpha=0.8)
185 ax.set_yticks(range(len(top10)))

```

```

186 ax.set_yticklabels(top10["特征"], fontsize=9)
187 ax.axvline(0, color="gray", linewidth=0.8)
188 ax.set_xlabel("逻辑回归系数", fontsize=11)
189 ax.set_title("Top 10 特征系数", fontsize=13, fontweight="bold")
190 if SAVE:
191     plt.savefig("figures/feature_importance.png", dpi=300)
192 plt.show()
193
194 # =====
195 # 9. 对比模型: 随机森林 (可选)
196 # =====
197
198 from sklearn.ensemble import RandomForestClassifier
199
200 rf = RandomForestClassifier(
201     class_weight="balanced",
202     n_estimators=100,
203     random_state=42,
204 )
205 rf.fit(X_train, y_train)
206
207 y_pred_rf = rf.predict(X_test)
208 y_prob_rf = rf.predict_proba(X_test)[:, 1]
209
210 auc_rf = roc_auc_score(y_test, y_prob_rf)
211 acc_rf = accuracy_score(y_test, y_pred_rf)
212
213 print(f"\n{' '*40}")
214 print("对比模型评估")
215 print(f"{' '*40}")
216 print(f"随机森林 - Accuracy: {acc_rf:.4f}, AUC: {auc_rf:.4f}")
217
218 # =====
219 # 10. 对比模型: XGBoost
220 # =====
221
222 from xgboost import XGBClassifier
223
224 neg_pos_ratio = len(y_train[y_train == 0]) / len(y_train[y_train == 1])
225
226 xgb = XGBClassifier(
227     scale_pos_weight=neg_pos_ratio,
228     n_estimators=100,
229     max_depth=5,
230     learning_rate=0.1,
231     random_state=42,
232     use_label_encoder=False,
233     eval_metric="logloss",
234 )
235 xgb.fit(X_train, y_train)
236
237 y_prob_xgb = xgb.predict_proba(X_test)[:, 1]
238 auc_xgb = roc_auc_score(y_test, y_prob_xgb)
239 acc_xgb = accuracy_score(y_test, xgb.predict(X_test))
240
241 print(f"\n{' '*40}")
242 print("XGBoost 评估结果")
243 print(f"{' '*40}")
244 print(f"XGBoost - Accuracy: {acc_xgb:.4f}, AUC: {auc_xgb:.4f}")

```

```

245
246 # ---- 10.1 三模型 ROC 曲线对比 ----
247 fpr_rf, tpr_rf, _ = roc_curve(y_test, y_prob_rf)
248 fpr_xgb, tpr_xgb, _ = roc_curve(y_test, y_prob_xgb)
249
250 fig, ax = plt.subplots(figsize=(7, 5.5))
251 ax.plot(fpr, tpr, color=C_LR, linewidth=2,
252         label=f"逻辑回归 (AUC = {auc:.3f})")
253 ax.plot(fpr_rf, tpr_rf, color=C_RF, linewidth=2,
254         label=f"随机森林 (AUC = {auc_rf:.3f})")
255 ax.plot(fpr_xgb, tpr_xgb, color=C_XGB, linewidth=2,
256         label=f"XGBoost (AUC = {auc_xgb:.3f})")
257 ax.plot([0, 1], [0, 1], "k--", linewidth=1, alpha=0.6)
258 ax.set_xlabel("假正率 (FPR)", fontsize=11)
259 ax.set_ylabel("真正率 (TPR)", fontsize=11)
260 ax.set_title("三模型 ROC 曲线对比", fontsize=13, fontweight="bold")
261 ax.legend(fontsize=10, loc="lower right")
262 ax.grid(alpha=0.3)
263 if SAVE:
264     plt.savefig("figures/roc_compare.png", dpi=300)
265 plt.show()
266
267 print("\n所有模型图片已保存至 figures/ 文件夹")
268
269 # =====
270 # 11. 逻辑回归模型验证
271 # =====
272
273 from sklearn.model_selection import cross_val_score
274 from sklearn.metrics import recall_score, precision_score
275
276 # ---- 11.1 5 折交叉验证 ----
277 cv_scores = cross_val_score(
278     lr, X_encoded, y, cv=5, scoring="roc_auc",
279 )
280 print(f"\n{' '*40}")
281 print("5 折交叉验证结果")
282 print(f"{' '*40}")
283 for i, s in enumerate(cv_scores):
284     print(f" 第 {i+1} 折 AUC: {s:.4f}")
285 print(f" 均值: {cv_scores.mean():.4f}")
286 print(f" 标准差: {cv_scores.std():.4f}")
287
288 # ---- 11.2 阈值敏感性分析 ----
289 print(f"\n{' '*40}")
290 print("阈值敏感性分析")
291 print(f"{' '*40}")
292
293 thresholds = np.arange(0.1, 0.85, 0.05)
294 recalls = []
295 precisions = []
296
297 for t in thresholds:
298     y_pred_t = (y_prob >= t).astype(int)
299     rec = recall_score(y_test, y_pred_t)
300     prec = precision_score(y_test, y_pred_t)
301     recalls.append(rec)
302     precisions.append(prec)
303     print(f" 阈值={t:.2f} Recall={rec:.3f} Precision={prec:.3f}")

```

```

304
305 fig, ax1 = plt.subplots(figsize=(6, 4.5))
306 ax1.plot(thresholds, recalls, "o-", color=C_CHURN, linewidth=2,
307         label="召回率 (Recall)")
308 ax1.plot(thresholds, precisions, "s-", color=BLUE, linewidth=2,
309         label="精确率 (Precision)")
310 ax1.axvline(0.5, color="gray", linestyle="--", alpha=0.5)
311 ax1.set_xlabel("判别阈值", fontsize=11)
312 ax1.set_ylabel("指标值", fontsize=11)
313 ax1.set_title("阈值敏感性分析", fontsize=13, fontweight="bold")
314 ax1.legend(fontsize=10)
315 ax1.grid(alpha=0.3)
316 if SAVE:
317     plt.savefig("figures/threshold_analysis.png", dpi=300)
318 plt.show()

```

SHAP 可解释性分析

Listing 4 shap_analysis.py ——SHAP 归因分析与可视化

```

1  import pandas as pd
2  import numpy as np
3  import matplotlib.pyplot as plt
4  import shap
5  import warnings
6  warnings.filterwarnings("ignore")
7
8  from style_config import *
9
10 # =====
11 # 1. 数据读入与清洗 (同 model.py)
12 # =====
13
14 df = pd.read_csv("Telco_customer_churn.csv")
15 df.drop(["CustomerID", "Count", "Country"], axis=1, inplace=True)
16 df["Total Charges"] = pd.to_numeric(
17     df["Total Charges"], errors="coerce"
18 ).fillna(0)
19
20 print("=== 数据清洗完成 ===")
21 print(f"行数: {df.shape[0]}, 列数: {df.shape[1]}")
22
23 # =====
24 # 2. 特征工程 (同 model.py)
25 # =====
26
27 drop_cols = [
28     "Churn Label", "Churn Score", "Churn Reason",
29     "Lat Long", "Latitude", "Longitude",
30     "State", "City", "Zip Code",
31 ]
32
33 X = df.drop(columns=drop_cols + ["Churn Value"])
34 y = df["Churn Value"]
35
36 cat_cols = X.select_dtypes(include=["object"]).columns.tolist()
37 num_cols = X.select_dtypes(include=[np.number]).columns.tolist()

```



```

38
39 X_encoded = pd.get_dummies(X, columns=cat_cols, drop_first=True)
40 print(f"编码后特征数: {X_encoded.shape[1]}")
41
42 # ---- 切分 + 标准化 ----
43 from sklearn.model_selection import train_test_split
44 from sklearn.preprocessing import StandardScaler
45
46 X_train, X_test, y_train, y_test = train_test_split(
47     X_encoded, y,
48     test_size=0.2,
49     stratify=y,
50     random_state=42,
51 )
52
53 num_cols_in_train = [
54     c for c in num_cols if c in X_encoded.columns
55 ]
56 scaler = StandardScaler()
57 X_train[num_cols_in_train] = scaler.fit_transform(
58     X_train[num_cols_in_train]
59 )
60 X_test[num_cols_in_train] = scaler.transform(
61     X_test[num_cols_in_train]
62 )
63
64 print("标准化完成")
65
66 # =====
67 # 3. 训练逻辑回归 (为 SHAP 提供模型)
68 # =====
69
70 from sklearn.linear_model import LogisticRegression
71
72 lr = LogisticRegression(
73     class_weight="balanced",
74     max_iter=1000,
75     random_state=42,
76 )
77 lr.fit(X_train, y_train)
78
79 print("逻辑回归训练完成")
80
81 # =====
82 # 4. SHAP 可解释性分析
83 # =====
84
85 explainer = shap.LinearExplainer(lr, X_train)
86 shap_values = explainer.shap_values(X_test)
87
88 X_test_df = X_test.copy()
89 try:
90     X_test_df.columns = [
91         c.replace("_", " ") for c in X_test_df.columns
92     ]
93 except Exception:
94     pass
95
96 # ---- 4.1 SHAP Summary Bar (Top 10 特征重要性) ----

```

```

97 fig, ax = plt.subplots(figsize=(7, 5))
98 shap.summary_plot(
99     shap_values, X_test_df, plot_type="bar",
100     max_display=10, show=False,
101 )
102 ax.set_title("SHAP 特征重要性 (Top 10)", fontsize=13,
103             fontweight="bold")
104 if SAVE:
105     plt.savefig("figures/shap_bar.png", dpi=300, bbox_inches="tight")
106 plt.close()
107
108 # ---- 4.2 SHAP 特征贡献 (水平柱状图) ----
109 mean_shap_all = np.abs(shap_values).mean(axis=0)
110 top_idx = np.argsort(mean_shap_all)[-10:]
111
112 fig, ax = plt.subplots(figsize=(7, 5))
113 ax.barh(
114     range(10),
115     mean_shap_all[top_idx],
116     color="#5A7B9A", alpha=0.85,
117 )
118 ax.set_yticks(range(10))
119 ax.set_yticklabels(
120     [X_test_df.columns[i] for i in top_idx],
121     fontsize=9,
122 )
123 ax.set_xlabel("平均 |SHAP| 值", fontsize=11)
124 ax.set_title("SHAP 特征重要性 (Top 10)", fontsize=13,
125             fontweight="bold")
126 ax.invert_yaxis()
127 if SAVE:
128     plt.savefig("figures/shap_be swarm.png", dpi=300,
129                 bbox_inches="tight")
130 plt.close()
131
132 # ---- 4.3 SHAP Dependence (Top 2 特征依赖图, 纯 matplotlib) ----
133 mean_shap = np.abs(shap_values).mean(axis=0)
134 top_idx = np.argsort(mean_shap)[-2:]
135
136 for idx in reversed(top_idx):
137     feat_name = X_test_df.columns[idx]
138     shap_col = shap_values[:, idx]
139
140     fig, ax = plt.subplots(figsize=(6, 4.5))
141     ax.scatter(
142         X_test_df.iloc[:, idx],
143         shap_col,
144         c=shap_col, cmap="RdBu_r",
145         alpha=0.5, s=10,
146     )
147     ax.axhline(0, color="gray", linewidth=0.8, linestyle="--")
148     ax.set_xlabel(feat_name, fontsize=11)
149     ax.set_ylabel("SHAP 值", fontsize=11)
150     ax.set_title(f"SHAP 依赖图 — {feat_name}", fontsize=13,
151                 fontweight="bold")
152     if SAVE:
153         fname = f"figures/shap_{feat_name[:15].replace(' ', '_')}.png"
154         plt.savefig(fname, dpi=300, bbox_inches="tight")
155     plt.close()

```

```

156
157 # ---- 4.4 打印 Top 5 关键因子 ----
158 mean_shap_df = pd.DataFrame({
159     "特征": X_test_df.columns,
160     "平均 |SHAP|": mean_shap,
161 })
162 mean_shap_df.sort_values(
163     "平均 |SHAP|", ascending=False, inplace=True
164 )
165
166 print(f"\n{' '*50}")
167 print("Top 5 关键流失因子")
168 print(f"{' '*50}")
169 for _, row in mean_shap_df.head(5).iterrows():
170     print(f" {row['特征']:30s} "
171           f"SHAP = {row['平均 |SHAP|']:.4f}")
172
173 print("\nSHAP 分析完成，图片已保存至 figures/ 文件夹")

```

客户风险分层与 ROI 计算

Listing 5 retention.py ——分位数分层与挽留策略 ROI 估算

```

1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import warnings
5
6 warnings.filterwarnings("ignore")
7
8 from style_config import *
9
10 # =====
11 # 1. 数据准备 (复用 model.py 的处理)
12 # =====
13
14 df = pd.read_csv("Telco_customer_churn.csv")
15 df.drop(["CustomerID", "Count", "Country"], axis=1, inplace=True)
16 df["Total Charges"] = pd.to_numeric(df["Total Charges"], errors="coerce").fillna(0)
17
18 drop_cols = [
19     "Churn Label",
20     "Churn Score",
21     "Churn Reason",
22     "Lat Long",
23     "Latitude",
24     "Longitude",
25     "State",
26     "City",
27     "Zip Code",
28 ]
29 X = df.drop(columns=drop_cols + ["Churn Value"])
30 y = df["Churn Value"]
31
32 cat_cols = X.select_dtypes(include=["object"]).columns.tolist()
33 num_cols = X.select_dtypes(include=[np.number]).columns.tolist()
34

```

```

35 X_encoded = pd.get_dummies(X, columns=cat_cols, drop_first=True)
36
37 from sklearn.model_selection import train_test_split
38 from sklearn.preprocessing import StandardScaler
39
40 X_train, X_test, y_train, y_test = train_test_split(
41     X_encoded,
42     y,
43     test_size=0.2,
44     stratify=y,
45     random_state=42,
46 )
47
48 num_cols_in_train = [c for c in num_cols if c in X_encoded.columns]
49 scaler = StandardScaler()
50 X_train[num_cols_in_train] = scaler.fit_transform(X_train[num_cols_in_train])
51 X_test[num_cols_in_train] = scaler.transform(X_test[num_cols_in_train])
52
53 # =====
54 # 2. 训练逻辑回归 + 预测全量流失概率
55 # =====
56
57 from sklearn.linear_model import LogisticRegression
58
59 lr = LogisticRegression(
60     class_weight="balanced",
61     max_iter=1000,
62     random_state=42,
63 )
64 lr.fit(X_train, y_train)
65
66 # 对训练集和测试集分别预测，合并
67 y_prob_train = lr.predict_proba(X_train)[: , 1]
68 y_prob_test = lr.predict_proba(X_test)[: , 1]
69 all_probs = np.concatenate([y_prob_train, y_prob_test])
70
71 # 合并完整的原始数据和预测概率
72 df_full = df.copy()
73 df_full["Churn_Prob"] = np.nan
74 df_full.loc[X_train.index, "Churn_Prob"] = y_prob_train
75 df_full.loc[X_test.index, "Churn_Prob"] = y_prob_test
76
77 print("=== 预测概率统计 ===")
78 print(f"均值: {all_probs.mean():.3f} 标准差: {all_probs.std():.3f}")
79 print(f"最小值: {all_probs.min():.3f} 最大值: {all_probs.max():.3f}")
80
81 # =====
82 # 3. 分层: 用分位数划分风险级别
83 # =====
84
85 p50 = np.percentile(all_probs, 50)
86 p80 = np.percentile(all_probs, 80)
87
88
89 def assign_tier(prob):
90     if prob > p80:
91         return "高风险"
92     elif prob > p50:
93         return "中风险"

```

```

94     else:
95         return "低风险"
96
97
98 df_full["Risk_Tier"] = df_full["Churn_Prob"].apply(assign_tier)
99
100 tier_counts = df_full["Risk_Tier"].value_counts()
101 tier_order = ["高风险", "中风险", "低风险"]
102
103 print(f"\n=== 分层阈值 ===")
104 print(f"低风险 {p50:.3f} < 中风险 {p80:.3f} < 高风险")
105 print(f"\n=== 各层级人数 ===")
106 total = len(df_full)
107 for t in tier_order:
108     cnt = tier_counts[t]
109     print(f" {t}: {cnt:5d} 人 ({cnt / total * 100:.1f}%)")
110
111 # ---- 3.1 分层饼图 ----
112 fig, ax = plt.subplots(figsize=(5, 4))
113 ax.pie(
114     [tier_counts[t] for t in tier_order],
115     labels=[f"{t}\n({tier_counts[t]}人)" for t in tier_order],
116     autopct="%1.1f%",
117     startangle=90,
118     colors=[C_HIGH, C_MID, C_LOW],
119     wedgeprops={"edgecolor": "white", "linewidth": 1.2},
120 )
121 ax.set_title("客户流失风险分层", fontsize=13, fontweight="bold")
122 if SAVE:
123     plt.savefig("figures/risk_tiers.png", dpi=300)
124 plt.show()
125
126 # =====
127 # 4. 各层级特征对比分析
128 # =====
129
130 key_metrics = {
131     "Tenure Months": "在网时长 (月)",
132     "Monthly Charges": "月消费金额 (美元)",
133     "Total Charges": "总消费金额 (美元)",
134     "CLTV": "生命周期价值",
135 }
136
137 tier_stats = []
138 for t in tier_order:
139     subset = df_full[df_full["Risk_Tier"] == t]
140     row = {"层级": t, "人数": len(subset)}
141     for col, _ in key_metrics.items():
142         row[col] = subset[col].mean()
143     # 关键分类特征的比例
144     row["光纤比例"] = (subset["Internet Service"] == "Fiber optic").mean()
145     row["月付合同比例"] = (subset["Contract"] == "Month-to-month").mean()
146     row["有家属比例"] = (subset["Dependents"] == "Yes").mean()
147     tier_stats.append(row)
148
149 tier_df = pd.DataFrame(tier_stats)
150
151 print(f"\n=== 各层级特征对比 ===")
152 print(tier_df.to_string(index=False))

```

```

153
154 # ---- 4.1 特征对比雷达图/柱状图 ----
155 compare_cols = [
156     "Tenure Months",
157     "Monthly Charges",
158     "光纤比例",
159     "月付合同比例",
160     "有家属比例",
161 ]
162 fig, axes = plt.subplots(1, 3, figsize=(9, 4))
163 colors = [C_HIGH, C_MID, C_LOW]
164
165 for i, col in enumerate(compare_cols):
166     vals = [tier_df[tier_df["层级"] == t][col].values[0] for t in tier_order]
167     axes[i % 3].bar(
168         tier_order,
169         vals,
170         color=colors,
171         alpha=0.8,
172         width=0.5,
173     )
174     axes[i % 3].set_title(col, fontsize=11, fontweight="bold")
175     axes[i % 3].tick_params(labelsize=8)
176
177 plt.tight_layout()
178 if SAVE:
179     plt.savefig("figures/tier_comparison.png", dpi=300)
180 plt.show()
181
182 # =====
183 # 5. 投入产出估算
184 # =====
185
186 high_risk_count = tier_counts["高风险"]
187 mid_risk_count = tier_counts["中风险"]
188
189 reach_rate = 0.60 # 触达率 (假设)
190 retain_rate = 0.30 # 挽留成功率 (假设)
191 cost_per_customer = 50 # 人均优惠成本 (美元)
192 avg_cltv = df_full["CLTV"].mean()
193
194 reached = high_risk_count * reach_rate
195 saved = reached * retain_rate
196 revenue_saved = saved * avg_cltv
197 total_cost = reached * cost_per_customer
198 roi = (revenue_saved - total_cost) / total_cost
199
200 print("\n== 投入产出估算 ==")
201 print(f"高风险客户数: {high_risk_count}")
202 print(f"触达率: {reach_rate:.0%} → 触达 {reached:.0f} 人")
203 print(f"挽回成功率: {retain_rate:.0%} → 挽回 {saved:.0f} 人")
204 print(f"人均 CLTV: ${avg_cltv:.0f}")
205 print(f"预期挽回收益: ${revenue_saved:,.0f}")
206 print(f"挽留总成本: ${total_cost:,.0f}")
207 print(f"投资回报率 (ROI): {roi:.2f} 倍")
208 print(f"净收益: ${revenue_saved - total_cost:,.0f}")
209
210 print("\n所有分层图片已保存至 figures/ 文件夹")

```

B 附录 2：建模过程的创新点说明

创新点一：三模型对比以实证手段验证线性假设

问题背景：逻辑回归因其系数可解释性，在需要向业务方解释建模结论时具有天然优势。但若特征间存在显著的非线性交互效应，线性假设将导致模型欠拟合，预测精度明显低于非线性模型。

创新做法：本文并未简单地“因为需要解释性所以选用逻辑回归”，而是通过逻辑回归、随机森林、XGBoost 三者的 AUC 对比（差值 < 0.01 ）和 ROC 曲线叠合，实证地论证了该数据集上线性假设的充分性。这一做法将模型选型的依据从主观偏好提升为可检验的实证结论。

创新点二：四层递进的探索性数据分析策略

问题背景：传统的 EDA 通常仅凭某一类统计量（如均值差异或相关系数）做出结论，缺乏多角度交叉验证。

创新做法：本文将 EDA 设计为“整体流失率 $\rightarrow$ 数值特征分布 $\rightarrow$ 分类特征流失率 $\rightarrow$ 相关性热力图”的四层递进结构：第一层建立问题全貌认知，第二、三层从数值和分类两个角度独立探索，第四层以相关系数的数值形式对前两层的定性发现进行量化锚定。此外，问题一的 EDA 结论在问题二的 SHAP 分析和问题三的分层画像中均有回扣（如在网时长的首位重要性与月付合同的高流失风险），使全文形成“探索 $\rightarrow$ 验证 $\rightarrow$ 应用”的闭环。

创新点三：类别不平衡的全流程应对

问题背景：客户流失数据中未流失客户占比 73.46%，流失客户仅占 26.54%，接近 1:2.8 的比例。

创新做法：本文从探索到建模对类别不平衡进行了系统性应对：

1. 数据探索阶段：报告整体流失率，明确类别不平衡程度；
2. 评估阶段：弃用对不平衡敏感的准确率，以 AUC 和 F1-score 为主要指标；
3. 训练阶段：对逻辑回归设置 `class_weight='balanced'`。

这一贯穿全流程的应对，保证了评估指标真实反映模型预警能力，而非给出“高准确率、零召回率”的虚假情况。

创新点四：分位数分层替代等距分层

问题背景：将客户分为高/中/低风险是制定差异化策略的前提。常见的等距切分（如 $[0, 0.33)/[0.33, 0.66)/[0.66, 1]$ ）在预测概率右偏分布下会导致中高风险层样本过于稀疏。

创新做法：采用 P50/P80 分位数作为切分点，既保证了三个层级都有充足的样本量（50%/30%/20%），又使层级间的特征差异足够显著：高风险层的月付合同比例达 99.9%、光纤比例 88.9%，与低风险层（19.5% 和 22.9%）形成鲜明对比。

C 附录 3：AI 工具使用说明

团队在完成本论文过程中使用了以下 AI 工具：

表 11 AI 工具使用清单

序号	工具	主要用途
1	DeepSeek-V4-Flash	通过 OpenCode Agent 辅助编写 Python 数据处理和可视化代码
2	DeepSeek-V4-Pro	查询模型原理知识，通过 WorkBuddy 辅助润色论文表达

AI 产出与人工修改

代码方面，AI 给出了 pandas 数据清洗、sklearn 建模和 matplotlib 绘图的基础框架，团队在此基础上补充了缺失值的业务判断逻辑、统一了图片配色与中文标签，并修正了分类变量的编码策略。论文文字方面，WorkBuddy 辅助调整了部分长句的断句和术语的统一表达，所有修改均经团队成员逐句审阅后采纳。

AI 错误记录

1. AI 建议用 SMOTE 过采样处理类别不平衡。团队注意到合成样本会引入虚假的 SHAP 特征贡献值，改用 `class_weight='balanced'` 仅调整损失权重。
2. AI 对合同类型等多分类名义变量使用了 Label 编码。团队指出”月付 =0，一年 =1，两年 =2”会引入虚假序关系，改为 One-Hot 编码。